

A Hybrid Multimodal Graph-RAG ChatBot with Inline Imagery

Team of Interns
Aditya , Abdualлах & Krupa

Under the Supervision of
Kajal Mishra, Gourav Garg, Priyanka Sharma,
Ravi, Suraj Bhatnagar Sir.



January to May 2026

EXECUTIVE SUMMARY

MedChat is a multimodal Graph Retrieval-Augmented Generation (RAG) assistant engineered to support cancer patients in interpreting clinical reports, understanding treatment protocols, and navigating dietary considerations. The system ingests peer-reviewed medical literature, extracts both textual and visual content, and serves context-grounded, citation-backed responses through a conversational interface.

Annually, 2.7 million people in India are diagnosed with Cancer, oncologist-to-patient ratio is 1:1600, meaning most patients never have adequate time with a specialist to understand their reports. Apps like these will surely be in demand.

The current implementation comprises a fully operational ingestion pipeline capable of processing **heterogeneous PDF formats** (scanned, vector, and mixed-layout), a **hybrid retrieval system** combining dense vector search (Qdrant) with **sparse keyword retrieval** (BM25) fused via Reciprocal Rank Fusion (**RRF**) and re-ranked using Maximal Marginal Relevance (**MMR**), and a generation layer powered by LLaMA 3.3 through the Groq API. The system includes web search fallback mechanism, frontend built with Streamlit along with image rendering and automated patient report analysis.

GraphRAG architecture, which uses Neo4j, allowing for structured reasoning over *cancer-chemotherapy-nutrition relationships; drug-food interaction detection* and biomarker-driven recommendation capabilities. This extension has the goal to build a knowledge graph of +600 nodes and +200 relationships, and route queries intelligently across the graph, vector and hybrid retrieval pathways.

TABLE OF CONTENTS

Contents	
CHAPTER 1	5
INTRODUCTION	5
1.1 BACKGROUND	7
1.2 MOTIVATION	7
1.3 SCOPE OF THE PROJECT	8
1.4 Limitations:	10
CHAPTER 2	12
PROJECT DESCRIPTION AND GOALS	12
2.1 LITERATURE REVIEW	12
PROJECT: Multi-modal Agentic RAG for Enterprise Relations	12
Model Choices, API Stack, and Compatibility Risks	16
A. Model/component choice table	16
B. Library/API stack — Role, strengths, weaknesses, risks and compatibility clue	17
2.2 RESEARCH GAPS IDENTIFIED	18
2.3 OBJECTIVES	19
2.4 PROBLEM STATEMENT	20
2.5 PROJECT PLAN	20
2. End-to-End Pipeline at a Glance	20
Challenges & Solutions	22
CHAPTER 3	23
TECHNICAL SPECIFICATION	23
3.1 REQUIREMENTS	23
GraphRAG Extension	23
3.1.1 Functional Requirements	24
3.1.2 Non-Functional Requirements	24
3.2 FEASIBILITY STUDY	25
Scalability & Production Readiness	25
3.3 SYSTEM SPECIFICATION	26
3.5 API ENDPOINTS	27
3.6 KNOWLEDGE BASE	28
CHAPTER 4	29
DESIGN APPROACH AND DETAILS	29

Tools & Technologies	29
4.1 SYSTEM ARCHITECTURE	30
4.2 SYSTEM DESIGN	31
List of Key Modules	31
Overview of the Workflow Diagram (Extract).....	31
4.2.1 Data Flow Diagram.....	32
4.2.2 ML Pipeline Design.....	32
CHAPTER 5.....	35
METHODOLOGY AND TESTING	35
5.1 METHODOLOGY	35
5.1.3 GraphRAG Extension	39
5.2 TESTING.....	42
Performance Considerations	42
CHAPTER 6.....	48
PROJECT DEMONSTRATION	48
6.1 Key Code Output logs for Project Report:	48
6.3 Demo Screenshots.....	50
CHAPTER 7.....	51
RESULT AND DISCUSSION	51
7.1 Model Performance	51
7.2 RAG System Efficacy.....	53
CHAPTER 8.....	56
CONCLUSION.....	56
REFERENCES.....	60
APPENDIX A – Sample Code.....	66

CHAPTER 1

INTRODUCTION

Our *RAG* journey has evolved from the learnings & applying phase to an exploratory developmental stage; wherein the product is evolving into a practical, modular, CPU and Cloud-aware pipeline.

The early weeks, focused upon- onboarding, environment setup, studying RAG loaders, comparing vector stores like Milvus vs FAISS vs ChromaDB, and understanding the workflow, architecture diagrams, etc. for which we relied on open-source benchmarking.

The organization gave us this project to attempt to upskill in multi modal RAG and how to create a hybrid retrieval engine and getting the images from the PDFs that are relevant to the context and the caregiver (*ASHA and Anganwadi workers*) then implements the same.

Patients frequently struggle to:

- Understand medical reports
- Interpret clinical terms
- Find reliable information about diseases

Current healthcare information systems have several limitations:

- Medical knowledge is scattered across research papers
- Existing chatbots often generate inaccurate responses.
- Traditional search systems lack contextual understanding
- Patients struggle to interpret clinical terminology

This project aims to build a GenAI-powered assistant that can answer healthcare queries using trusted medical knowledge sources.

Healthcare needs intelligent systems to identify high-risk patients proactively and give clinicians immediate evidence-based answers to complex medical questions. While traditional EWS have been valuable, they are predominantly based on limited sources of structured data and do not capture crucial clinical signals that remain buried in unstructured physician notes, nursing assessments, and discharge summaries. Additionally, clinicians spend excessive amounts of time manually searching through clinical guidelines, laboratory references, and extensive patient histories — time that could be better spent directly caring for patients and making decisions regarding their care.

Retrieval-augmented generation (RAG) components for clinical question answering is another innovation of the system. The module uses Sentence-BERT embeddings to encode clinical guidelines (ACC/AHA, ESC, WHO), medical references, and anonymized patient records into dense vector representations. These vector representations are indexed using FAISS (facebook ai similarity search) for efficient similarity retrieval at scale. When clinicians pose *natural language queries*, the system retrieves contextually relevant passages and grounds the response from a large language model (LLM) in verified medical knowledge — therefore minimizing hallucinations and providing properly cited and trustworthy medical facts.

Patients with cancer struggle with understanding multimodal images — (CT, MRI, Xray) medical reports and fragmented healthcare information. This system bridges the gap between patients and caregivers (via radiologist). Simpler for any strata of patients to understand as it is now consolidated and secure.

This **proof-of-concept** demonstrates the feasibility of combining predictive analytics with *knowledge-augmented generation* to create a unified clinical intelligence platform that enhances patient safety, reduces clinician cognitive burden, and supports evidence-based medical decision-making.

1.1 BACKGROUND



The company, being a client facing - one requires all the internal teams to be at least aware of AI-ML and *a team of three Interns*, were tasked with helping facilitate the required training. Learning the first month on multi model pipeline and its 6 phases (*Ingestion, Chunking, Embedding, Retrieving, Augmenting and Generating*). After having learnt all the phases, we then were given the liberty to, with certain set limitations in the hardware and software space to choose any topic of our own choice.

We went ahead with **healthcare** as the domain specifically leaning towards *oncology*, i.e. cancer. We were cognizant with the fact that as being and utilizing low computational power systems and the lack of availability of costly peer reviewed medical documents, we were stuck to using only certain open-source libraries and in-silico for our data set.

The modern healthcare landscape generates vast amounts of data, ranging from **structured Electronic Health Records (EHRs)** to **unstructured clinical notes, lab reports, and imaging**. Most of this information was in isolation making early detection and deterioration of health of the patient very difficult. Traditional methods rely on linear data rules which fail to capture non liner patient data.

Due to this, the burden of getting accurate data is falling on healthcare professionals. LLMs can offer solutions for providing information, but system sometimes hallucinates so we need for more grounding mechanisms.

1.2 MOTIVATION

Using the first informer(s) as well as having access to large volumes of healthcare-related data; the tool was developed to create a simple, safe, *HIPAA compliant* pipeline for both providers and patients. The product could take doctors' reports and transcribed medical image reports and break them down into one report. Through its ability to make complicated health record data easily available, understandable, and navigable, it makes health-related decisions easier for clinicians.

Need for Explainable, Interactive AI:

Clinicians want a risk score but also want to know why a certain decision should be made. In order to turn a passive predictive analytics tool into an active clinical assistant that provides answers to questions such as "*what is the standard hypertension treatment plan for my diabetic patient?*".

1.3 SCOPE OF THE PROJECT

Mostly, Patients & sometimes, Caregivers, struggle to understand the medical reports & clinical terminology. Medical knowledge is fragmented across thousands of research institutions. Even medical chatbots, hallucinate due to a lack of ground knowledge of the eating habits of the said patients which has adverse side effects to the chemotherapeutic drugs that have being prescribed to the cancer patients. This leads to the vicious cycle wherein, even though the patient and caregiver are obedient and follow the prescribe schedule.

In particular, social workers like *ASHA and Anganwadi workers* are frontline community health volunteers under the NHM, acting as a crucial interface between rural/urban communities and public health systems like AIIMS. They can overlook their own eating habits which then leads to the adverse effect in spite of medication.

In our use case, we went ahead with oncology (*in this case, top 6 common cancer reviewed PDFs*), clinical report complexity, pathology reports, staging documents, and treatment plans are written in dense medical terminology.

Keyword: Trustworthiness and Citation Deficit, Fragmented Information Landscape, search misses' relational context (e.g., **Drug** → **Side Effect**→ **Food**)

Proposition:

- ✓ Hybrid retrieval engine RAG + GraphDB assistant grounded in peer-reviewed medical literature
- ✓ Structured knowledge graph (Neo4j) for relationship-aware reasoning
- ✓ Patient report → personalized, context-aware answers

MedChat addresses the identified limitations through a vertically integrated, multimodal RAG architecture purpose-built for oncology-related patient support. The solution encompasses:

Current System (Implemented)

- **Multimodal Ingestion Pipeline.** Processes medical research PDFs using Docling as the primary parser with PyMuPDF fallback, handling scanned (OCR), vector, and mixed-layout documents. Extracts text with multi-stage cleaning and normalization, performs header-aware semantic chunking, classifies content by cancer type and content category (clinical, prognosis, statistics), extracts images with multi-layer noise filtering, and grounds images within text via `[IMAGE: filename.png]` tags.
- **Hybrid Retrieval System.** Combines dense retrieval via Neo4j [earlier Qdrant] (using BAAI/bge-base-en-v1.5 embeddings [3]) with sparse retrieval via BM25.

$$\alpha = B\beta + V\mu \quad [\text{Eq.1}]$$

Retrieval results are fused using Reciprocal Rank Fusion (**RRF**) and re-ranked with Maximal Marginal Relevance (**MMR**) to balance relevance and diversity.

- **Context-Grounded Generation.** Employs LLaMA 3.3 via the Groq API to produce responses grounded strictly in retrieved context, with explicit source citations and patient-accessible language.
- **Web Search Fallback.** Activates DuckDuckGo-based web search when the RAG pipeline yields insufficient results, synthesizing external information through the LLM with appropriate safety disclaimers.
- **Automated Patient Report Analysis.** Parses uploaded clinical reports to extract diagnosis, stage, treatment plan, and clinical findings, then cross-references extracted entities against the ingested literature corpus.
- **Interactive Frontend.** A Streamlit-based interface providing chat functionality, PDF upload, inline image rendering, and session-persistent conversation history.
- **Neo4j Knowledge Graph.** Models structured relationships between cancer types, chemotherapy drugs, drug interactions, eating-related side effects, and nutrition guidelines across 600+ nodes and 200+ relationships [1][5].

- **Intelligent Query Routing.** Directs queries to graph-based, vector-based, or hybrid retrieval pathways based on query characteristics.
- **Structured Reasoning.** Supports Cypher-based querying for drug–food–interaction reasoning, protocol-level insights, and biomarker-driven dietary recommendations [2][5].

Knowledge base: Melanoma, Breast, Lung Cancer, Leukaemia & Osteosarcoma Medical PDFs.
inSilico synthetically generated (*claude opus 4.6*)

1.4 Limitations:

- ✓ The system relies on synthetic data (generated based on freely available data) and public datasets (NIH, NICC, etc) to ensure privacy.
- ✓ Real-time processing capabilities are dependent on underlying hardware.

Limitation	Description
Text-Only Retrieval	Most healthcare chatbots and QA systems operate exclusively on textual data, ignoring figures, histopathology images, and diagnostic visuals embedded in literature.
No Source Attribution	Consumer-facing health tools (e.g., symptom checkers, general chatbots) generate responses without citing specific sources, undermining clinical trustworthiness.
Single Retrieval Strategy	Systems relying solely on dense vector retrieval miss keyword-specific matches; those relying solely on keyword search miss semantic equivalences.
No Report-Level Analysis	Existing tools do not ingest and automatically analyse patient-specific clinical reports against a curated knowledge base.
Static Knowledge Bases	Many systems use fixed FAQ datasets or pre-curated content that cannot be extended with new research without significant re-engineering.
No Relational Reasoning	Flat retrieval architectures cannot model or traverse structured relationships, such as drug–drug interactions, drug–food contraindications, or protocol hierarchies.

Generic Response Generation	Responses are not tailored to cancer-specific contexts such as staging, biomarker profiles, or treatment-phase-dependent dietary restrictions.
-----------------------------	--

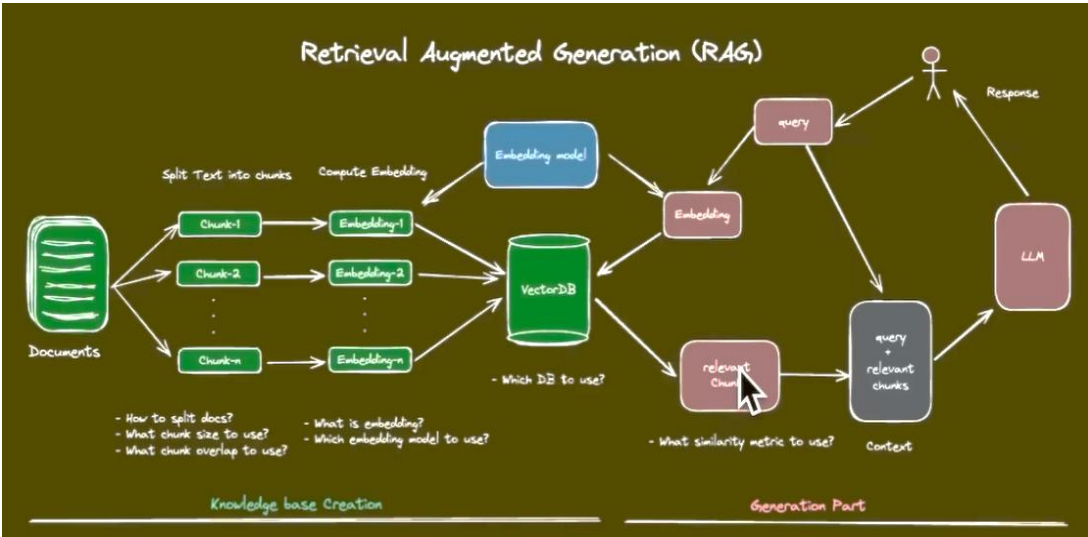


figure: RAG- a state transition diagram showing the steps from Knowledge base creation to Generation modal

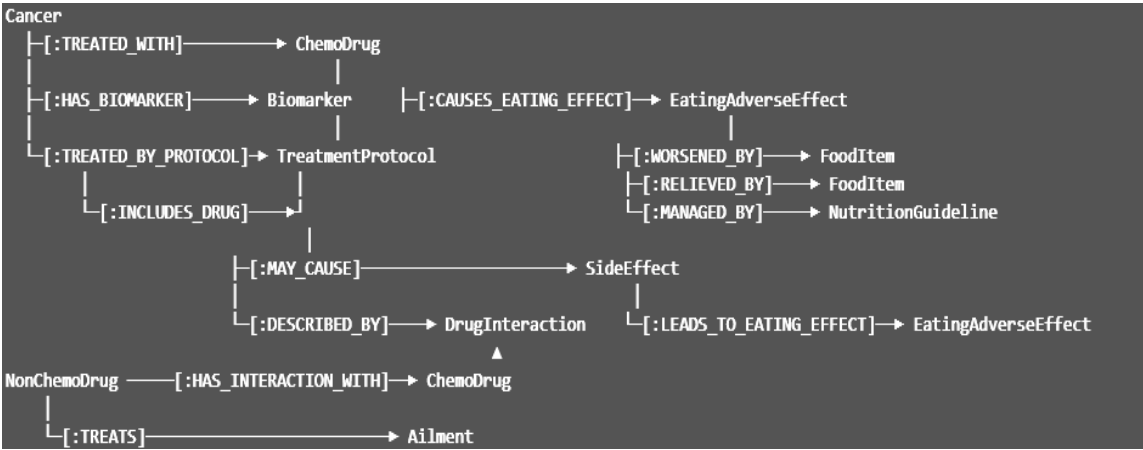


figure: work flow diagram enumerating – the implicit relationship between, chemo and non-chemo drugs, their side effects and how they are mediated by eating habits of a patients {cancer–chemotherapy–nutrition relationships}

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

PROJECT: Multi-modal Agentic RAG for Enterprise Relations

- Data Extraction: Successfully implemented and tested document loading using PyMuPDF and Docling (Jan 27th).
- R&D & Learning:
- Initiated research into various Chunking Strategies (Fixed-size, Recursive, Semantic) to optimize data retrieval for the RAG pipeline (Jan 30th).
- Explored the nuances of handling complex PDFs via Docling to ensure high-fidelity data extraction.
- Strategic Initiative: Company-wide GEN-AI Upskilling
- Resource Curation: Browsed through 200+ external resources over a two-day sprint to identify world-class, free, and industry-relevant AI/ML training materials.
- Submission: Successfully compiled and submitted the "AI/ML Training External Links" document to Priyanka Ma'am before the deadline (Jan 29th).

Chunking as Information Design

Chunking is much more than simply limiting tokens to meet formatting requirements.

Chunking is the art of preserving structural integrity as well as meaning when dividing a body of content into manageable units.

Adaptive Document Chunking: Structured content is the type of content that would include code snippets, tables, or markdown documents. Adaptive chunking preserves structured content in a manner that retains its original meaning. As such, adaptive chunking maintains the relationship between data elements and increases the efficiency of retrieval.

Timestamp-Based Chunking: With respect to conversational-type data coming from sources such as slack messages, system logs or chat transcripts, the typical method of chunking tends to lose sight of how conversations occur naturally. Conversations are contextual; i.e. typically, the relevance of a particular message is contingent upon previous messages and their timestamp. Timestamp-based chunking groups messages together based on specific time intervals (for example, every five minutes or per session). Therefore, each group of messages retains the contextual continuity which makes these groups more useful for retrieval and analysis purposes.

Chunking strategy comparison for learning-

Strategy	Best for	Strength	Weakness	Puzzle clue
Fixed size	Uniform or simple text	Simple and fast	Can cut meaning mid-thought	Easy to implement, risky for context
Recursive	General documents	Balanced and practical	Still blind to deeper semantics	Good default when structure exists
Structure-aware	Markdown, technical docs, code, sections	Preserves headings/sections well	Depends on clean formatting	Great when document structure is extractable
Semantic	Topic-shifting long docs	Best meaning preservation	Costlier and more complex	Use when topics shift rapidly
Adaptive / Hybrid	Irregular enterprise docs	Context-aware and robust	Harder to tune	Powerful, but only if evaluation is disciplined

- **Pipeline Optimization** via Inline image inference system to ground and give confidence
- Implemented a lightweight pipeline using **Docling**, **PyMuPDF**, and **Moondream/ Qwen2-vl:3b** to ensure efficient performance on CPU-based environments (Feb 3).

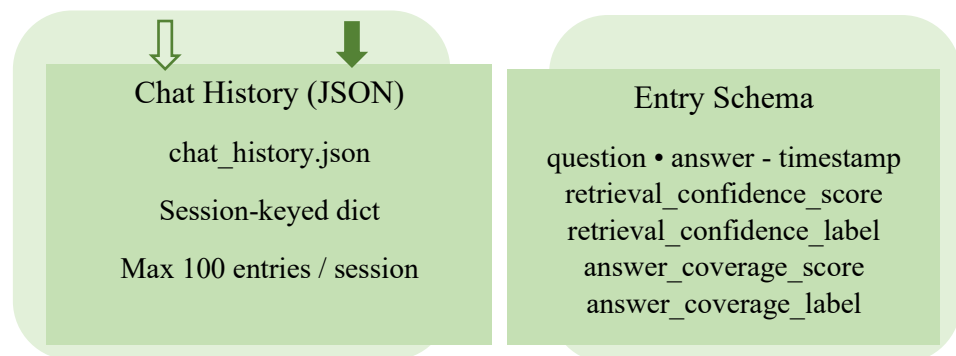


figure: Comparison of various chunking strategies are being compared on two varying parameters – top k results and the alpha value i.e. sum of k(Keyword) and v(Vector)

Retriever	Wins when...	Loses when...	Takeaway
BM25	Exact words matter (codes, acronyms, IDs)	User wording differs from document wording	Essential safety net; do not remove lightly.

Semantic / Vector	Intent and synonym matching matter	Exact lexical precision is critical	Powerful but can be fuzzy on exact terms.
Hybrid	Queries mix exact terms and broad intent	Implementation is sloppy or score fusion is weak	Best default for messy enterprise/document workflows.
Strategy	Best Performance On...	CPU Impact	Overall Verdict
BM25	Exact codes, acronyms, and technical IDs.	Negligible	Essential safety net.
Semantic	Broad questions, "How-to" intent.	Medium	Good, but needs reranking.
Hybrid	Multi-topic PDFs and complex queries.	Medium	Winner for enterprise docs.
MMR	Long PDFs with repetitive info.	Low	Best for summarization tasks.

- Experiment with a lightweight embedding stack using **Ollama** ([sentence-transformer/nomic-embed-text](#)) to optimize the pipeline for resource-constrained devices (Feb 3rd).
- **Image Processing:** Successfully extracted ~90 images from complex documents, generating brief AI descriptions for each (Feb 3rd).
- **Adaptive Chunking:** Integrated the **Hybrid Chunker** library from Docling for adaptive chunking, following guidance from Kajal Ma'am to improve retrieval precision (Feb 4th).
- **Embedding Strategy:** Developed a workflow to store AI-generated image descriptions in lists for text-based chunking and embedding. Successfully embedded ~20 images using **embedding gemma** (Feb 5th).

GraphRAG RETRIEVAL PATTERNS

The core GraphRAG pattern is very simple, yet powerful.

1. Do a vector search to find an initial set of nodes
2. Traverse the graph around those nodes to add context
3. (Optional:- Rank the outcome using the graph & pass the top-k documents to the LLM)

2.1.1 Guidelines & Risk Frameworks

Model Choices, API Stack, and Compatibility Risks

When one goes about architecting their project, they must not only keep the technological specifications in mind but also how each model (be it a ML one or other) and API (RESTful or not) mingle with each other! What is implied by this analogy is that the Compatibility (due to native kernel/low level or due to versioning/high level) dissimilarities are to be conveyed very clearly to each founding engineer.

They then are to, being cognizant of the risks involved, put forward a set of guidelines that then goes on to become the framework of whole company and dictates the models the choose and the API they call and the embedded systems they design on & for.

A. Model/component choice table

Component area	Explored in findings	Current POC/code	Why this matters
Text extraction	Docling + PyMuPDF experiments	Docling preferred, PyMuPDF fallback	Combines richer structure with operational reliability.
OCR	Needed for difficult/scanned content	EasyOCR optional	Useful but not allowed to block the whole pipeline.
Embeddings	Ollama/nomic-embed-text and other	BAAI/bge-base-en-v1.5 in current code	Shows a shift from exploration to a concrete,

	lightweight options were explored		normalized embedding setup.
Vector store	Milvus vs FAISS were studied	Qdrant client local store in current code	Vector backend choice affects retrieval ergonomics and metadata filtering behavior.
Sparse retrieval	BM25 emphasized in search lab	BM25Retriever in current code	Critical lexical safety net.
Answer model	Multiple options were explored during R&D	Groq-backed llama-3.3-70b-versatile in retrieval layer	Shows separation between retrieval logic and answering model.
Vision / multimodal	Moondream, Qwen2-VL, CLIP, and image-description workflows were explored	Current code emphasizes extracted captions and image tags	Practical CPU-aware multimodality can start with textualized visuals.

B. Library/API stack — Role, strengths, weaknesses, risks and compatibility clue

Library / API stack piece	Role in pipeline	Strengths	Trade-offs / risk	Compatibility clue from code
Docling	Structured PDF extraction	Better structure, tables, and richer document understanding	Optional install and not guaranteed on every machine	Code falls back when Docling is unavailable or sparse
PyMuPDF (fitz)	Reliable PDF text/image extraction	Practical, direct, dependable fallback	Layout structure is less semantically rich than Docling	Acts as baseline extractor when needed
EasyOCR	OCR for scanned pages	Helps rescue unreadable/scanned text	Optional dependency; startup/performance overhead	Only used if installed
LangChain package family	Chunking, docs, retrievers, vector abstractions	Fast prototyping and composability	Package split means version drift can hurt imports	Code imports multiple packages: core, community, qdrant, huggingface

Qdrant client	Vector storage and metadata filtering	Local store + vector search + filter support	Older installs may lack filter classes or behaviors	Code explicitly guards for filter import availability
duckduckgo_search	Web fallback search	Useful fallback when local context fails	Optional dependency; not core retrieval path	Code disables fallback search if package unavailable
Groq API	Answer generation and follow-up generation	Fast hosted inference pattern	External dependency and API-key reliance	Used only after retrieval has already grounded context

2.2 RESEARCH GAPS IDENTIFIED

Patients many times struggle to:

- Understand medical reports
- Interpret clinical terms
- Find reliable information about diseases
 - The information is fragmented between research papers, dietary advice, and treatment guides.
- Current healthcare information systems have several limitations:
 - Medical knowledge is scattered across research papers
 - Existing chatbots often generate inaccurate responses.
 - Traditional search systems lack contextual understanding
 - Patients struggle to interpret clinical terminology.

These factors collectively create a significant gap between the information available in medical literature and the actionable understanding accessible to cancer patients.

This project aims to build an AI-powered assistant that can answer healthcare queries using trusted medical knowledge sources.

2.3 OBJECTIVES

The primary objectives of the project are:

- Build a Healthcare Question Answering System.
- Implement Retrieval Augmented Generation (RAG).
- Integrate GraphDB for relationship-based retrieval.
- Improve answer accuracy using medical knowledge sources.
- Enable users to ask questions related to diseases, drugs and treatments.
- Reviewed provided resource materials.
 - Milvus vs FAISS
 - HIPAA (Privacy) and FHIR (Interoperability)
- Explored Python libraries specific to our stack as well as brushing up on fundamentals.
- Started exploring Document Loaders and RAG pipelines to understand data ingestion flows.

Strategic Initiative: Company-wide AI Upskilling

- **Course Design:** Collaborated on the curriculum design to equip all existing Aziro developers at all levels with AI capabilities.
- **Submission:** Submitted the "Courses for various roles and levels" proposal to Priyanka Ma'am (Jan 21st).

Project: Multi-modal Agentic RAG for Enterprise Relations

- **Pipeline Optimization:**
 - Implemented a lightweight pipeline using Docling, PyMuPDF, and Moondream/Qwen2-vl:3b to ensure efficient performance on CPU-based environments (Feb 3rd).
 - Experiment with a lightweight embedding stack using **Ollama** ([sentence-transformer/nomic-embed-text](#)) to optimize the pipeline for resource-constrained devices (Feb 3rd).

- **Image Processing:** Successfully extracted ~90 images from complex documents, generating brief AI descriptions for each (Feb 3rd).
- **Adaptive Chunking:** Integrated the **HybridChunker** library from Docling for adaptive chunking, following guidance from Kajal Ma'am (project's team lead) to improve retrieval precision (Feb 4th).
- **Embedding Strategy:** Developed a workflow to store AI-generated image descriptions in lists for text-based chunking and embedding. Successfully embedded ~20 images using **embedding gemma** (Feb 5th).
- Started exploring **OpenAI's CLIP** model for advanced image processing and retrieval within the multimodal RAG framework (Feb 6th).
- Refine the image-to-text-to-embedding pipeline to reduce processing time (currently 2+ hours for 20 images).

2.4 PROBLEM STATEMENT

Healthcare professionals are currently overwhelmed by **high-volume, high-velocity patient data**, leading to "alert fatigue" and missed deterioration signals. Current manual workflows for risk assessment are reactive rather than proactive. Furthermore, accessing specific medical protocols requires navigating dense PDF documents with various file formats that are hard to comprehend, consuming valuable time. There is a critical need for an automated, intelligent system that can solve the same.

2.5 PROJECT PLAN

Cancer patients struggle to understand multimodal (images – RT, MRI, X-Ray) medical reports and fragmented healthcare information. This system bridges the gap between patient and caregivers.

Simpler for any patient to understand

2. End-to-End Pipeline at a Glance

High-level flow

1. Documents enter the ingestion layer.
2. Text is extracted using Docling when available; otherwise PyMuPDF acts as the practical fallback.
3. Scanned pages can optionally go through OCR.
4. Images and tables are extracted, filtered, and turned into caption-aware chunks.
5. Cleaned text is chunked with section awareness and overlap.
6. Embeddings are created and pushed into Qdrant client.
7. At query time, dense retrieval and BM25 run in parallel.
8. RRF fuses both result lists.
9. MMR reduces redundancy in the final context window.
10. The answer model responds with citations to retrieved chunks; if context is insufficient, a web-search fallback can be triggered.

Architecture layers

Layer	What it does	Why it exists
Ingestion	Extracts, cleans, tags, chunks, embeds, and stores source material.	Without reliable ingestion, retrieval quality collapses regardless of model quality.
Retrieval	Fetches evidence via dense + sparse search and fuses results.	No single retrieval method is consistently best across exact-match, synonym, and long-form queries.
Reranking	MMR selects diverse yet relevant chunks.	Prevents wasting context on repeated information.
Generation	Builds answers from retrieved evidence.	Turns evidence into usable responses while keeping answers grounded.
Learning Layer	Puzzle-driven modules, demo prompts, and decision journal.	Transforms the POC into a repeatable internal learning asset.

Challenges & Solutions

Challenge	Solution Implemented
Heterogeneous PDF formats	Dual-parser architecture (Docling + PyMuPDF fallback) with format-specific preprocessing pathways for scanned, vector, and mixed-layout documents.
Medical text normalization	Multi-stage cleaning pipeline tuned for medical text artifacts: ligature expansion, encoding normalization, and clinical notation preservation.
Image noise in extracted figures	Multi-layer filtering pipeline applying resolution thresholds, blank detection, and decorative element removal.
Caption extraction inconsistency	Primary extraction with spatial proximity analysis, supplemented by fallback strategies for non-standard caption formats.
Balancing precision and recall in retrieval	Hybrid dense+sparse retrieval with RRF fusion provides complementary coverage; MMR re-ranking ensures result diversity.
LLM hallucination risk	Strict context-grounding in the system prompt constrains the LLM to retrieved content. Source citations provide verifiability.
Insufficient internal knowledge coverage	Web search fallback with DuckDuckGo provides external context, with explicit safety disclaimers differentiating external from internal sources.
Medical terminology accessibility	System prompt directs the LLM to accompany medical terms with patient-friendly explanations.
Multimodal response coherence	Image grounding via [IMAGE] tags ensures visual content is contextually integrated rather than randomly appended.
Embedding model selection for medical domain	BAAI/bge-base-en-v1.5 selected for its strong general-domain performance and manageable resource footprint [3]; domain-specific fine-tuning identified as a future optimization.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

Current Stack

Category	Technology	Purpose
Document Parsing	Docling	Primary PDF parser with layout analysis
Document Parsing	PyMuPDF	Fallback PDF parser
OCR	Integrated via Docling	Scanned PDF text extraction
Vector Database	Qdrant	Dense vector storage and similarity search [4]
Sparse Retrieval	BM25 (rank-bm25)	Keyword-based sparse retrieval
Web Search	DuckDuckGo (duckduckgo-search)	Fallback external search
Frontend	Streamlit	Web-based user interface
Language	Python 3.x	Primary implementation language

GraphRAG Extension

Category	Technology	Purpose
Graph Database	Neo4j	Knowledge graph storage and traversal
Query Language	Cypher	Structured graph querying [5]

Integration	Neo4j Python Driver	Graph database connectivity
Vector Store	Neo4j	Hybrid retrieval vector component [2]

3.1.1 Functional Requirements

Requirement	Details
Hybrid Retrieval	The system must retrieve medical context by executing hybrid searches across vector storage (e.g., Chroma) and graph databases (e.g., Neo4j) to formulate a relevant context.
Clinical Q&A Generation	The system must process natural language queries alongside patient reports to generate clinical answers, explicitly referencing extracted document sources.
Multimodal Alignment	The system must extract image captions embedded within the retrieved context and evaluate their semantic relevance to both the user query and the final generated answer.
Web Search Fallback	The system must automatically trigger an external web search (e.g., DuckDuckGo) when the local vector and graph contexts lack meaningful content or are entirely empty.
Automated Evaluation	The system must compute RAGAS and TruLens metrics, generating structured JSON reports that detail accuracy, composite scores, and letter grades for each pipeline execution.

3.1.2 Non-Functional Requirements

Requirement	Details
Latency Tracking	The evaluation pipeline must capture and log the precise execution time (in milliseconds) for each generation cycle without causing significant operational overhead.
Privacy & Local Inference	The system must support entirely local data processing and evaluation scoring. LLM-as-a-judge capabilities must be configurable to run on-device via Ollama (e.g., llama3.1:8b) to prevent exposure of sensitive medical data.

Reliability & Graceful Degradation	The evaluation frameworks must maintain continuous operation. If the LLM system is unavailable or fails, the system must automatically revert to heuristic, Python evaluation metrics.
Medical Safety Compliance	The system must evaluate generated responses against pre-defined serious medically prescribed patterns (e.g., advising patients not to pursue treatments either guaranteeing total cures) and apply various possible penalties for violations, if any.
Portability	The criteria for the evaluation of the modules, must be such that they are self-contained, by utilizing only the Python standard library & NumPy for heuristic scoring, all the more, ensuring the system which can be deployed without hefty external machine learning frameworks.

3.2 FEASIBILITY STUDY

Scalability & Production Readiness

Current State Assessment

Dimension	Current State	Production Requirement
Deployment	Single-instance Streamlit application	Containerized deployment (Docker) behind a load balancer
Vector Database	Single Qdrant instance	Qdrant cluster with replication and sharding
LLM Access	Direct Groq API calls	API gateway with rate limiting, retry logic, and fallback providers
Authentication	None (prototype)	Role-based access control (RBAC)
Monitoring	Application-level logging	Centralized logging, metrics dashboards, and alerting
Data Privacy	Local processing	HIPAA-compliant data handling, encryption at rest and in transit

Compliance Considerations

This being a healthcare domain, deployment has certain non-negotiables:

- **HIPAA Compliance:** The Privacy Rule standards address the use and disclosure of individuals' PHI by entities subject to the rule. These individuals and organizations are called "covered entities."
- **Medical Disclaimer Framework:** Response generation must consistently include appropriate medical disclaimers, which the current system already implements.
- **Data Retention Policies:** Patient-uploaded reports must be subject to configurable retention and deletion policies.

Type	Assessment
Technical Feasibility	The stack (Python, Docling, embedding and generative models) is mature and open-source. Ollama enables local LLM inference without cloud dependencies with Groq (as a fallback) as well as Qdrant have good free tier limits. Podman works for containerization.
Economic Feasibility	The project utilizes entirely open-source software (Apache 2.0, MIT licenses). No licensing costs. Optional cloud deployment on Azure container apps in future.
Social Feasibility	By providing clinical guidance/awareness support and reducing administrative burden, the system can positively impact patient safety and reduce clinician workload.

3.3 SYSTEM SPECIFICATION

Hardware Requirements

Environment	Specification
Development	Intel Core i5, 8GB RAM (min), 50GB storage. Ollama requires 8GB+ RAM for Llama2. CUDA-enabled GPU (optional for acceleration).
Deployment	Podman-capable host (Docker compose) with 8GB+ RAM for LLM inference.

Software Stack

Layer	Technology	Version	Purpose
Backend	Python	3.11+	Runtime
	Uvicorn	≥0.22.0	ASGI server
	Pydantic	≥2.0.0	Request/Response validation
	NumPy	≥1.24.0	Numerical operations
	Sentence-Transformers	≥2.2.0	Text embeddings (all-MiniLM-L6-v2)
	FAISS-CPU	≥1.7.0	Vector similarity search
LLM	Ollama	Latest	Local LLM inference server
	Llama2	7B	Answer generation for RAG
Frontend	Streamlit	≥1.26.0	Web UI dashboard
DevOps	Podman	20.10+	Containerization
	Docker Compose	2.x	Multi-container orchestration
	Git	2.x+	Version control

3.5 API ENDPOINTS

In-line images during the augmentation stage of RAG relevant image with a similar title is fetched from Local path and displayed to the end user. This enables grounding by providing images.

The local path of the image being fetched is displayed as well (file name); this works in tandem with the pre-existing workflow of the pipeline giving a nested final answer for the caregiver.

A final gracious fallback in the form of DuckDuckGo is provided as a sanity check saying that our vector and graph databases (AuraDB) are limited.

3.6 KNOWLEDGE BASE

The RAG system indexes **6 peer reviewed oncology review papers** forming a categorized as follows:

Document Name	What it Includes
lung-cancer-review.pdf	An overview of ALK-targeted therapies and resistance mechanisms in non-small cell lung cancer.
melanoma-skin-cancer-review.pdf	Updated overview of cutaneous melanoma, from molecular biology and epidemiology to clinical diagnosis and treatment strategies.
osteosarcoma-review.pdf	Summarizes the clinical types, diagnostic procedures, and multi-modal treatment options for osteosarcoma
skin-cancer-types-review.pdf	Global comparative analysis of skin cancer subtypes and their prevalence across different continents.
acute-leukemia-review.pdf	Current insights into the pathophysiology, classification, and management of acute lymphoblastic leukemia.
breast-cancer-review.pdf	Evaluates the potential and effectiveness of localized topical and transdermal therapies for treating breast cancer.

CHAPTER 4

DESIGN APPROACH AND DETAILS

Tools & Technologies

What our POC taught us?: It taught us not only the well-spoken about hard skills for career growth as is evident in this report but also the soft skills which are supple to pick up on and fragile to manage but when done right work wonders for be it a team, a company, or the organisation as a whole!

As, we progressed through the weeks of development - we moved from basic loading of document to extraction of data using PyMuPDF and Docling. Post this, we experimented with multimodal extractions and image processing.

The present priority for ingestion script is as follows:-

Docling would be first preferred after with PyMuPDF will be used as backup

OCR is optionally used for scanned pages and image noise if any would be filtered using *multiple heuristics*, captions are changed into separate data chunks

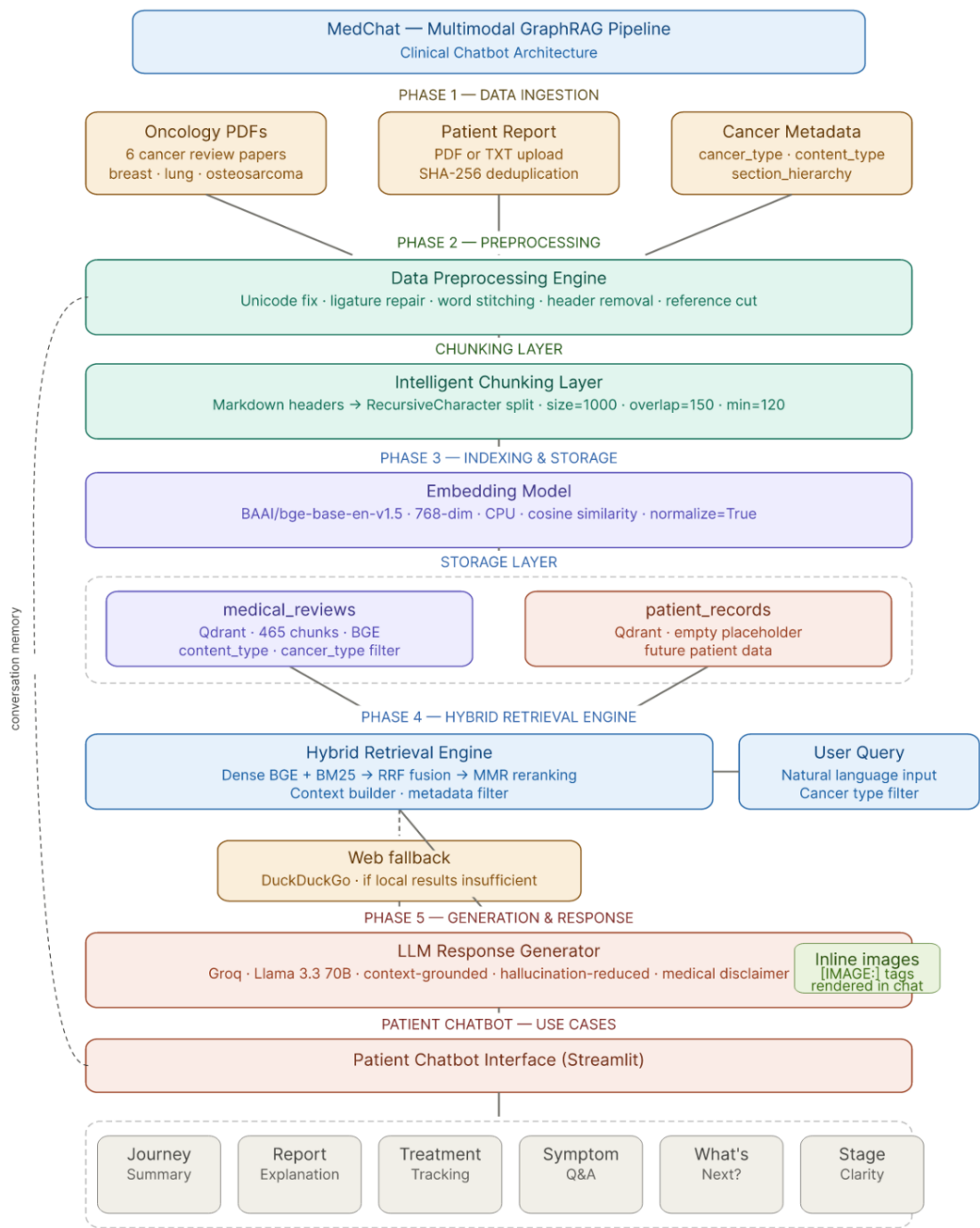
And visuals are injected back into text with file name tags.

Current ingestion decision map

Decision point	Chosen behavior	Reason
Text extraction	Docling first, PyMuPDF fallback	Better structure, when possible, dependable fallback when not.
Scanned pages	EasyOCR only if available	Keeps OCR optional instead of blocking the pipeline.
Images	Extract, deduplicate, filter noise, keep meaningful assets	Reduces junk from logos/badges/QR-like artifacts.
Captions	Store as separate caption chunks	Makes figure/table evidence retrievable even when raw image understanding is absent.

Image references	Inject [IMAGE: filename] tags into markdown	Let's the answer layer call visual assets explicitly.
------------------	---	---

4.1 SYSTEM ARCHITECTURE



System Architecture

System Architecture Diagram: - denoting phase 1-5, taking it further till the caregivers

4.2 SYSTEM DESIGN

List of Key Modules

Overview of the Workflow Diagram (Extract)

- **patients.csv**: Providing the ground truth for *AJCC/Enneking stages* and treatment plans (*MAP chemotherapy to lifestyle*).
- **reports_index.csv**: Acting as the lookup table to link specific MRI/CT text files to patient-IDs.

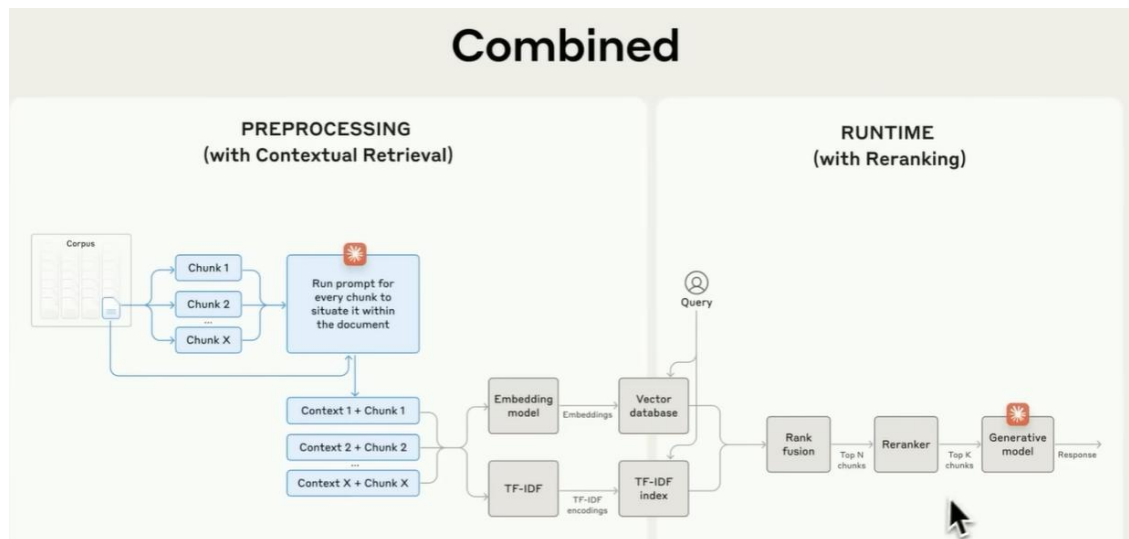
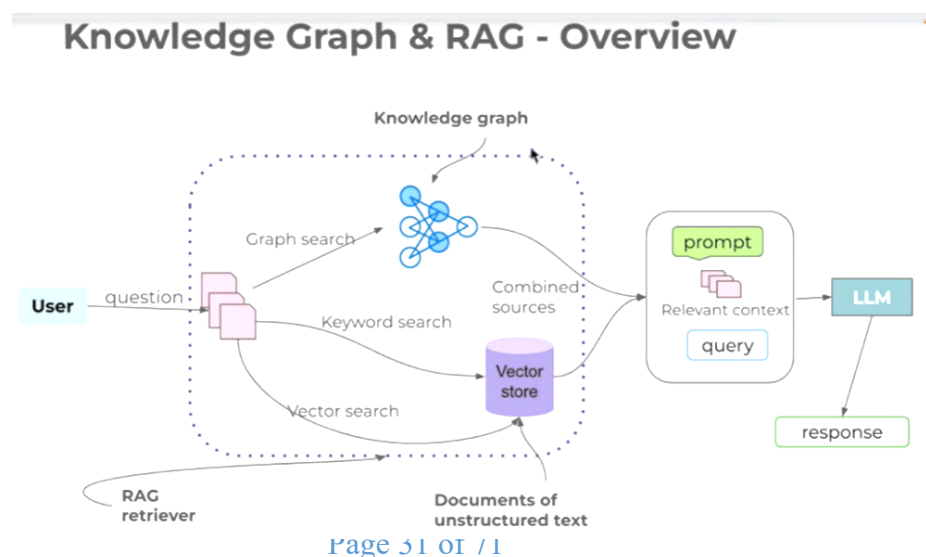
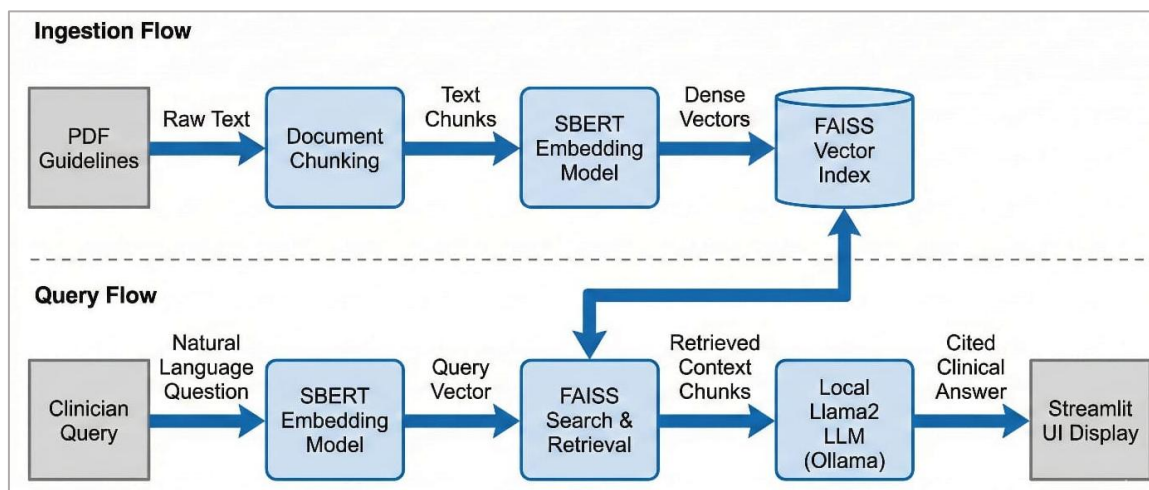


figure:- When a contextual trunk retrieval is modified at vectorDB, influenced by user queries then the RRF provides richer answers via Gen AI modal.



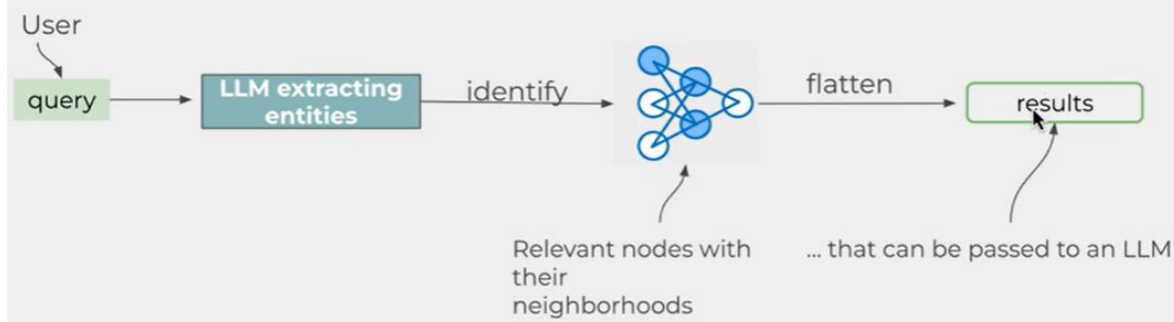
4.2.1 Data Flow Diagram



4.2.2 ML Pipeline Design

- **Retrieval Innovation:** have successfully implemented a "RAG Search Lab" that compares traditional keyword search (BM25) against modern Semantic and Hybrid methods.
- **MMR Integration:** By adding **Maximal Marginal Relevance (MMR)**, the pipeline now accounts for diversity in results, preventing the LLM from receiving redundant information from similar chunks.
- **UI Visualization:** The new dashboard provides a clear visual breakdown of scores (e.g., BM25 vs. Vector Semantic), which is vital for explaining technical performance to mentors like Sagar Sir (upper management) or Ravi / Nikitha (points of contact).

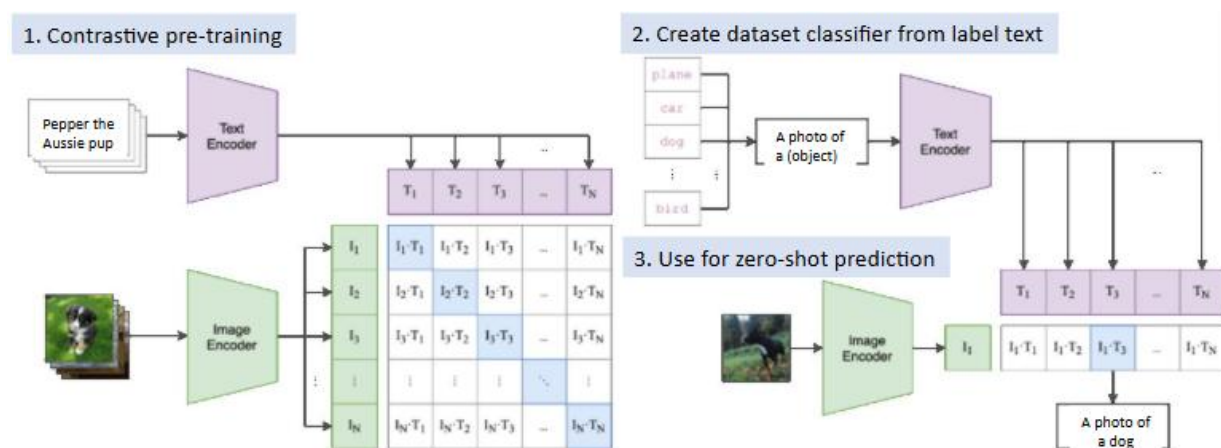
Graph Retriever - Flow



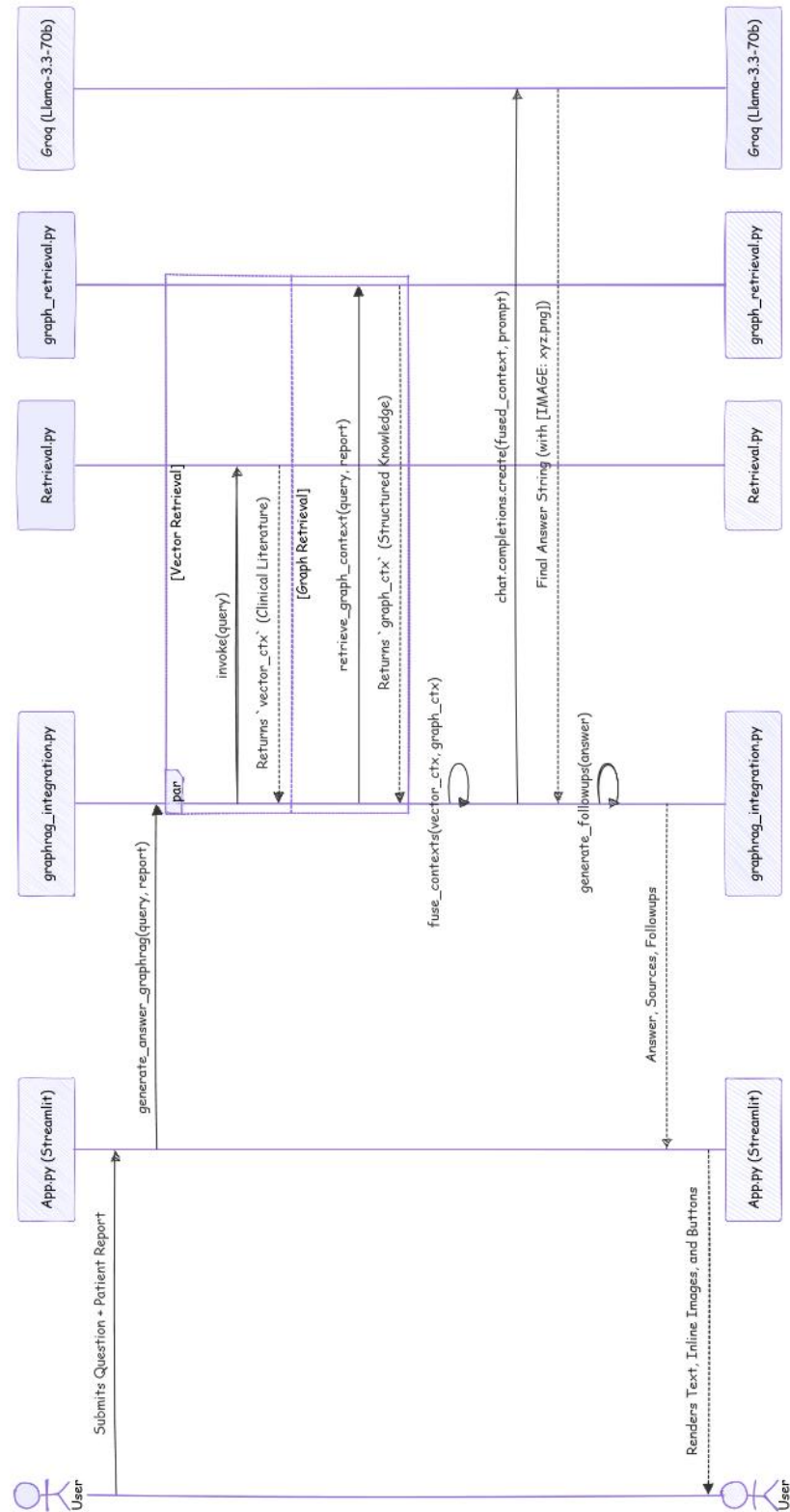
4.2.3 Risk Classification Thresholds

Started exploring **OpenAI's CLIP** model for advanced image processing and retrieval within the multimodal RAG framework (Feb 6).

Refine the image-to-text-to-embedding pipeline to reduce processing time (currently 2+ hours for 20 images).



4.2.4 Sequence Diagram



CHAPTER 5

METHODOLOGY AND TESTING

5.1 METHODOLOGY

Development Approach

The project follows an iterative development approach with modular components.

Algorithm 1: Medical Chat Application User Interface (App.py)

Objective: To provide an interactive interface for users to upload medical reports, receive automated clinical analysis, and chat with a medical RAG assistant.

Input: User text query, User uploaded PDF/text report

Output: Analysed report summary, streaming clinical answers, referenced images, Suggested follow-up questions

Steps:

1. Start
2. Initialize Streamlit session state variables and apply custom UI themes.
3. Read input data from file uploader or chat input box.
4. If a new patient report is uploaded, compute its hash and run automated clinical analysis.
5. If a user query is submitted, perform retrieval augmented generation (RAG) to generate a streaming answer.
6. Update chat history with user inputs and assistant responses.
7. Extract and display relevant inline image citations.
8. Generate and display contextual follow-up questions for the user.
9. Stop

Algorithm 2: RAG Evaluation Metrics Computation (Evaluation/feedbacks.py)

Objective: To quantitatively evaluate retrieval quality, answer alignment, web-search fallback correctness, and multimodal image relevance.

Input: User query, LLM generated answer, Vector context, retrieved text chunks, Selected image captions

Output: Individual and composite evaluation scores (0.0 to 1.0)

Steps:

1. Start
2. Initialize tokenization utilities, stop-words list, and punctuation filters.
3. Read query, generated answer, contexts, and image captions.
4. Calculate retrieval quality by computing context richness, query alignment, and chunk coverage.
5. Calculate answer quality by computing context utilization and query completeness.
6. If the web fallback marker is present in the answer, evaluate whether the fallback triggered correctly based on context emptiness.
7. Else, assign full fallback correctness credit assuming standard RAG succeeded.
8. Calculate multimodal relevance by computing the cosine similarity between the selected image captions and the query/answer.
9. Aggregate all sub-scores into a final composite evaluation score dictionary.
10. Generate output.
11. Stop

Algorithm 3: Knowledge Graph Schema Bootstrapper (Synth KG/graph_schema.py)

Objective: To define and idempotently apply node labels, constraints, and indexes to the Neo4j oncology knowledge graph.

Input: Neo4j database connection client

Output: Bootstrapped Neo4j database schema with verified constraints and indexes

Steps:

1. Start
2. Initialize connection client to the Neo4j graph database.
3. Read predefined node labels, relationship types, and schema configurations.

4. Loop through uniqueness constraints array and apply CREATE CONSTRAINT IF NOT EXISTS.
5. Loop through full-text index definitions and apply CREATE FULLTEXT INDEX IF NOT EXISTS.
6. Loop through vector index configurations and apply CREATE VECTOR INDEX IF NOT EXISTS.
7. If an error occurs (e.g., due to an older Neo4j version), log a warning and skip the index.
8. Generate console logs indicating successful schema bootstrap.
9. Stop

5.1.1 Data Acquisition

Explored Snythea and MACCROBAT but finally went ahead by creating a static knowledge base for our vector DB due to the resource as well as time constraints working on big data and fine-tuning models weren't feasible. The top 6 cancers we found to be in our research are as follows:

1. Lung
2. Melanoma
3. Acute Lymphoblastic Leukemia
4. Breast
5. Osteosarcoma
6. Skin pathogenesis

These are from renowned sources such as: Elsevier, BJBMS, Cancers MDPI, Open review.

When it came to specific patient healthcare records, we saw the sample templates and generated in-silico data for the same containing general aspects of their condition.

This was done for benchmarking purposes.

The user himself can upload his/her report in a .pdf or .txt format.

5.1.2 Feature Engineering

Implemented Features

Feature	Description
Multimodal PDF Ingestion	Processes scanned, vector, and mixed-layout PDFs with OCR support, extracting both text and images.
Semantic Chunking	Header-aware chunking preserving section hierarchy and document structure.
Cancer-Type Classification	Automatic classification of ingested content by cancer type for targeted retrieval.
Image Extraction & Grounding	Extracts images with noise filtering, associates' captions, and grounds images in text via [IMAGE] tags.
Dual Embedding Strategy	Generates separate embeddings for text content and image captions using BAAI/bge-base-en-v1.5 [3].
Hybrid Retrieval (Dense+Sparse)	Combines Qdrant dense vector search with BM25 sparse retrieval for comprehensive result coverage [4].
RRF Fusion	Merges ranked results from dense and sparse retrieval using Reciprocal Rank Fusion.
MMR Re-ranking	Applies Maximal Marginal Relevance to balance relevance with diversity in final results.
Context-Grounded Generation	LLaMA 3.3 generates responses strictly grounded in retrieved context with explicit citations.
Patient Report Analysis	Automatically parses uploaded clinical reports, extracting diagnosis, stage, and treatment plan.
Web Search Fallback	DuckDuckGo-based external search with LLM synthesis when internal retrieval is insufficient.
Safety Disclaimers	Automatically appends medical safety disclaimers to generated responses.
Session-Based Chat History	Maintains conversation context within user sessions for coherent multi-turn interactions.

5.1.3 GraphRAG Extension

Feature	Description
Knowledge Graph Construction	Neo4j-based graph modelling cancer–drug–nutrition relationships [1][5] stored in AuraDB.
Intelligent Query Routing	Automatic routing to graph, vector, or hybrid retrieval pathways based on query semantics.
Cypher-Based Querying	Structured graph queries for drug interactions, contraindications, and protocol relationships [2][5].
Drug–Food Interaction Reasoning	Traversal-based reasoning over drug–food interaction pathways in the knowledge graph.

Preprocessing Pipeline: isn't required as all the data are either peer reviewed papers or inSilico datasets using references.

5.1.4 Model Development

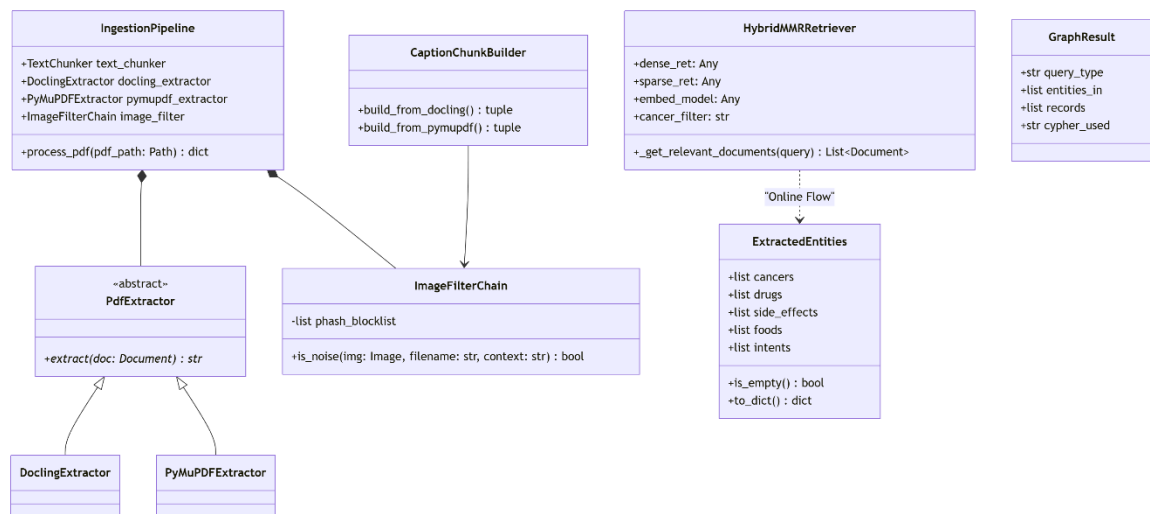
Embedding Model	BAAI/bge-base-en-v1.5	Text and caption embedding generation [3]
LLM	LLaMA 3.3 (via Groq API)	Response generation

Initial benchmark summary

Metric	Score	Interpretation
Context Recall	1.000	All ground-truth claims were covered by the retrieved context.
Context Precision	0.900	Most retrieved chunks were relevant to the answer.
Faithfulness	0.667	Two-thirds of answer claims were grounded in the retrieved context.
Answer Correctness	0.538	Moderate factual overlap with the ground truth answer.

RAGAS Score (Overall)	0.513	Balanced aggregate of the four core RAGAS metrics.
Answer Relevance	0.274	The answer only weakly tracked the original question.
Noise Sensitivity	0.800	The answer remained stable under noisy context perturbation.

5.1.4 CLASS Diagram



RAG (Retrieval-Augmented Generation) Workflow

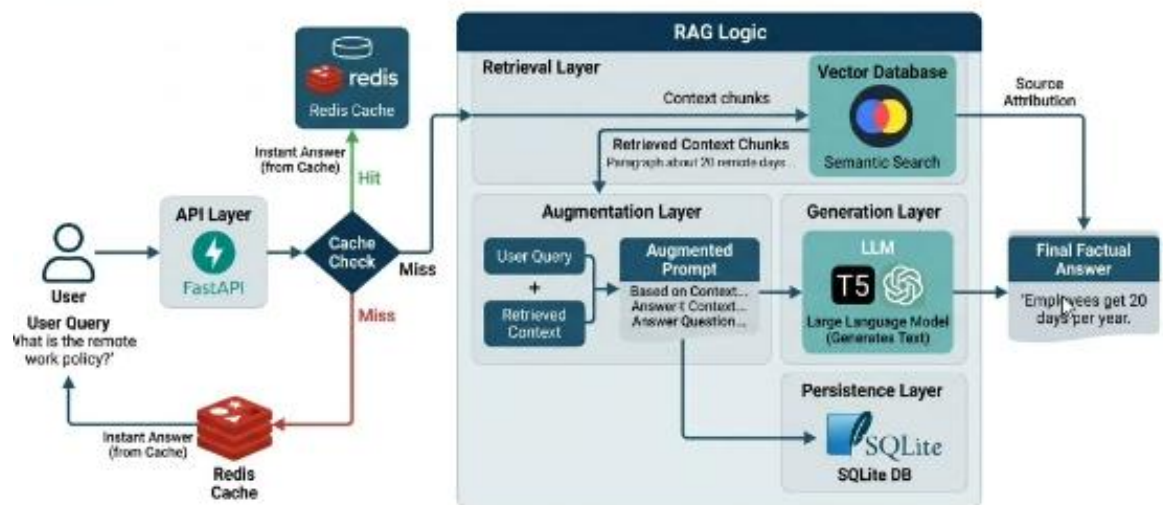


figure: Proposed Sequence diagram with future Redis caching (Hit/Miss) as well as a persistence layer via (PostgreSQL instead) to provide the user a factual answer [with graceful fallback when necessary]

5.1.5 RAG Prompt Template:

Goal 1: Chemo Drugs to Food Habits

This query **Maps Drug -> CAUSES -> SideEffect <- HELPS/WORSENS <- Food**.
It establishes exactly what patients should or shouldn't eat based on their specific chemotherapy side effects.

Example Output:

[Drug: Cisplatin]

- > Causes Nausea | [+] HELPS with Banana (bland easy-to-digest BRAT food)
- > Causes Nausea | [+] HELPS with White Rice (low-fiber easily digestible)
- > Causes Nausea | [+] HELPS with Toast (bland starchy food)
- > Causes Nausea | [+] HELPS with Crackers (bland carbohydrate buffer)
- > Causes Nausea | [-] WORSENS with Fried Foods (high fat delays gastric emptying)
- > Causes Nausea | [-] WORSENS with Alcohol (direct gastric mucosal irritant)

Goal 2: Different Chemo Drugs Affecting Cancer

This query maps **Cancer -> TREATED_WITH -> Drug -> CAUSES -> Side-Effect**.
It establishes how different drug protocols affect various cancers and outlines the specific side effects a patient would encounter based on their line of treatment.

Example Output:

[Cancer: Osteosarcoma]

--- Treated with Cisplatin (Protocol: MAP, Line: first) ---

- > Causes Appetite Loss [Severity: MODERATE]
- > Causes Taste Changes [Severity: MODERATE]
- > Causes Peripheral Neuropathy [Severity: MODERATE]
 - > Causes Nausea [Severity: SEVERE]
 - > Causes Vomiting [Severity: SEVERE]
 - > Causes Nephrotoxicity [Severity: SEVERE]
 - > Causes Ototoxicity [Severity: SEVERE]

5.2 TESTING

Performance Considerations

Requirement	Details
Latency Tracking	The evaluation pipeline must capture and log the precise execution time (in milliseconds) for each generation cycle without causing significant operational overhead.
Privacy & Local Inference	The system must support entirely local data processing and evaluation scoring. LLM-as-a-judge capabilities must be configurable to run on-device via Ollama (e.g., llama3.1:8b) to prevent exposure of sensitive medical data.
Reliability & Graceful Degradation	The evaluation frameworks must maintain continuous operation. If the LLM judge backend becomes unavailable or fails, the system must automatically gracefully revert to heuristic, pure-Python scoring metrics.
Medical Safety Compliance	The system must evaluate generated responses against predefined dangerous medical advice patterns (e.g., advising patients to stop treatments or guaranteeing cures) and apply severe numeric penalties for any violations.
Portability	The evaluation modules must be self-contained, utilizing only the Python standard library and NumPy for heuristic scoring, ensuring the system can be deployed without heavy external machine learning frameworks.

5.2.1 Ingestion Performance

Metric	Consideration
DF Processing Throughput	Docling provides higher accuracy but lower throughput than PyMuPDF. The fallback architecture ensures processing completes even for problematic documents, at the cost of reduced structural fidelity in fallback cases.
Image Extraction Overhead	Multi-layer noise filtering adds processing time per document but significantly reduces storage waste and downstream retrieval noise from irrelevant images.
Embedding Generation	BAAI/bge-base-en-v1.5 provides a balance between embedding quality and inference speed. Batch embedding is used during ingestion to maximize GPU utilization.

5.2.2 Retrieval Performance

Metric	Consideration
Query Latency	Dense retrieval via Qdrant operates in sub-second latency for the current corpus scale. BM25 sparse retrieval adds marginal latency. RRF and MMR post-processing are computationally lightweight.
Retrieval Quality	Hybrid retrieval consistently outperforms single-modality retrieval in internal evaluations, particularly for queries mixing specific medical terms with general-language descriptions.
Scalability	Qdrant supports horizontal scaling via sharding. BM25 indices can be partitioned by cancer type for sub-linear scaling with corpus growth.

5.2.3 Generation Performance

Metric	Consideration
--------	---------------

Inference Latency	Groq API provides low-latency LLaMA 3.3 inference via custom hardware (LPU). Typical response generation completes within 2–5 seconds.
Context Window Utilization	Context assembly manages chunk selection to maximize information density within the LLM's context window, prioritizing higher-relevance chunks.
Response Quality	Context grounding and citation requirements in the system prompt trade marginal creativity for significant improvements in factual accuracy and verifiability.

5.2.4 Unit Testing

Test Case	BM25 (Keyword)	Semantic (Vector)	Hybrid (Combined)
Specific Code (e.g., "Error 503")	Excellent. Finds exact matches.	Poor. Might match "Error" in general.	Best. Keeps the exact match at the top.
Synonyms (e.g., "login assistance")	Poor. Won't find "authentication guide."	Excellent. Understands they mean the same.	Balanced. Finds the synonym and the exact word.
Short Queries (e.g., "passcode")	Strong. Finds every instance.	Vague. Might find too much context.	Strong. Filters for relevance.
Long Questions	Noisy. Gets distracted by words like 'the'.	Strong. Focuses on the core "intent."	Best. Filters noise while keeping keywords.

5.2.5 Integration Testing

Test Case	Expected Behavior
End-to-End Ingestion Flow	Raw Medical PDF -> Text & Image Extraction -> Embedding Generation -> Successfully stored in Qdrant Vector DB
Patient Report Auto-Analysis	Upload Report (PDF/Text) -> App Trigger -> LLM returns structured summary (Diagnosis, Stage, Treatment)
RAG Query Flow	Clinical Question -> Hybrid Retrieval (Vector + BM25) -> LLM answer with inline medical images & source citations
Evaluation Pipeline Flow	RAG query outputs -> TruLens/Ragas API -> Quantitative scoring (Context Relevance, Groundedness) generated

5.2.6 Sample Test Patients

The system includes 3 calibrated test patients for demonstration:

Use Case 1: Dietary Guidance During Treatment

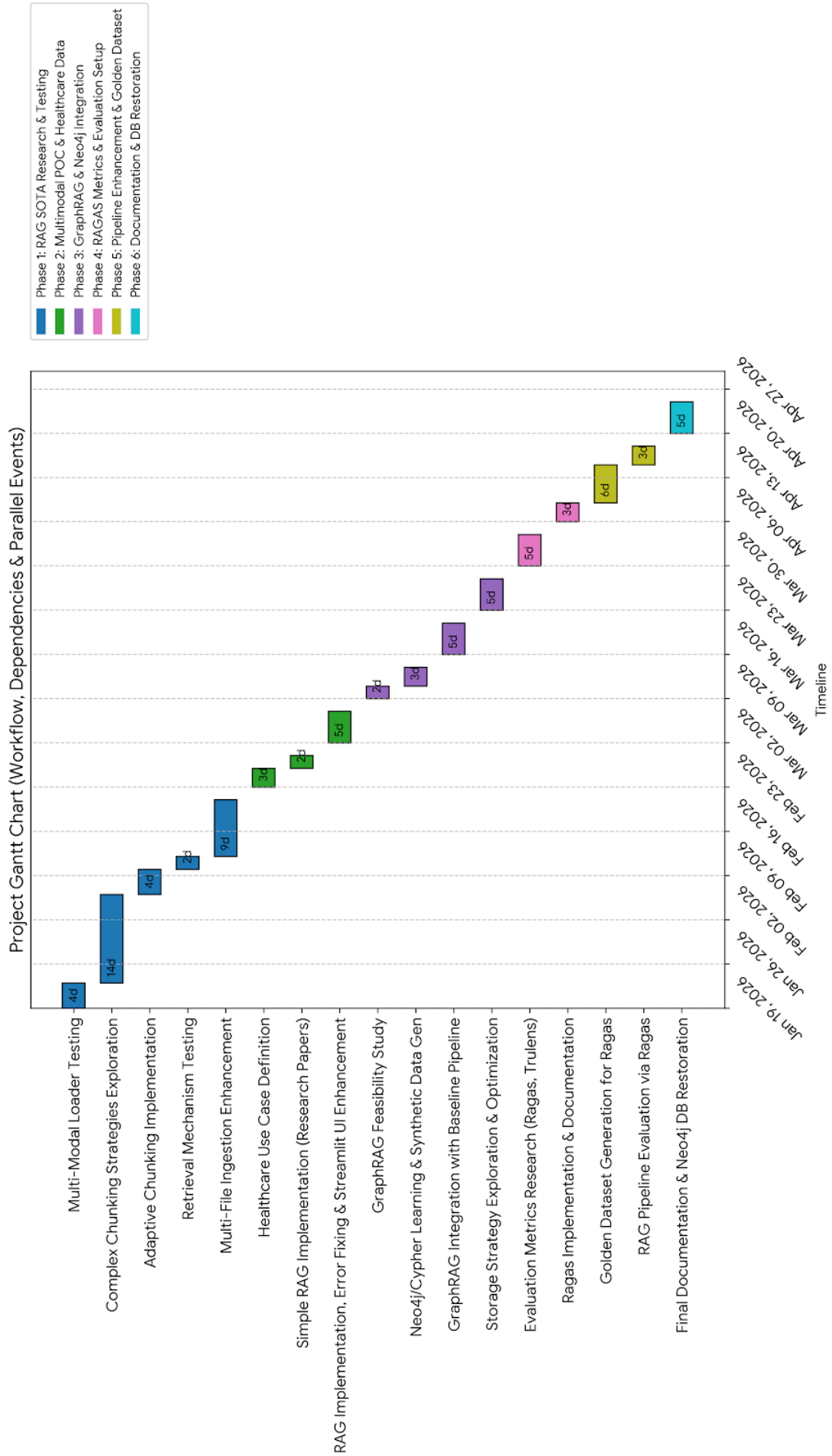
Field	Detail
Actor	Cancer patient undergoing chemotherapy
Trigger	Patient asks, "What should I eat during my second cycle of chemotherapy?"
Process	<i>Current system:</i> Retrieves dietary guidance from ingested literature. <i>Future (GraphRAG):</i> Routes to graph pathway, traverses drug–food interaction nodes, identifies contraindications, and generates a dietary plan grounded in graph-encoded nutritional guidelines.
Output	Evidence-based dietary recommendations, with drug–food interaction warnings (GraphRAG) and citations.

Use Case 2: Drug Interaction Inquiry (GraphRAG)

Field	Detail
Actor	Cancer patient or healthcare professional
Trigger	Query: "Does methotrexate interact with grapefruit?"
Process	Query router directs to graph pathway. Cypher query traverses' drug–food interaction edges. Results are augmented with literature evidence from the vector pathway if available.
Output	Structured interaction report with severity classification, mechanism explanation, and supporting references.

Use Case 3: Multi-Turn Conversational Support

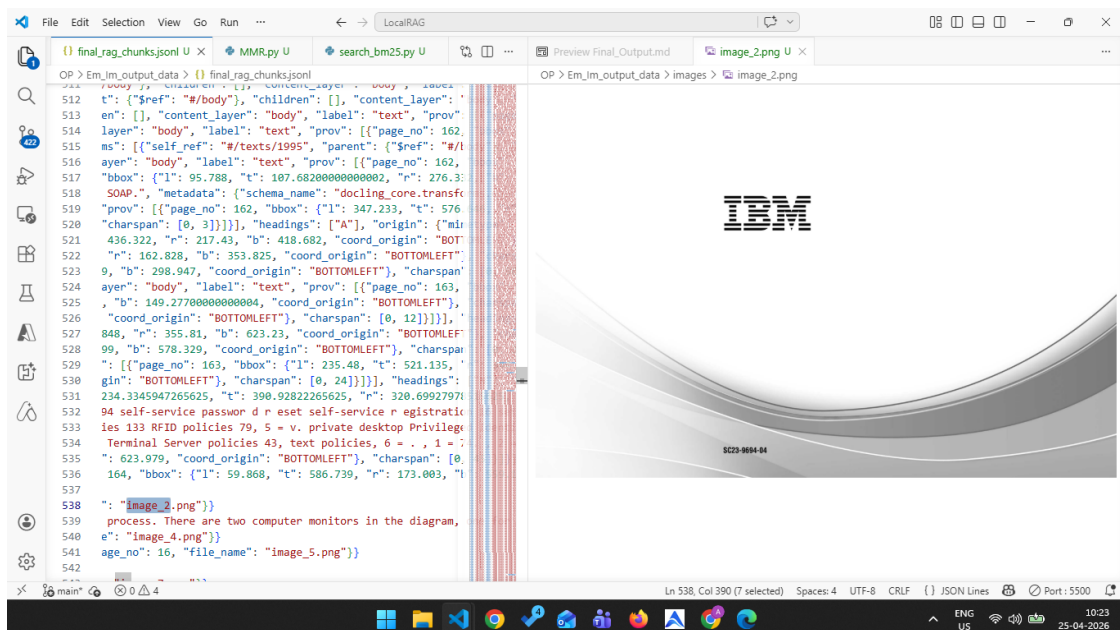
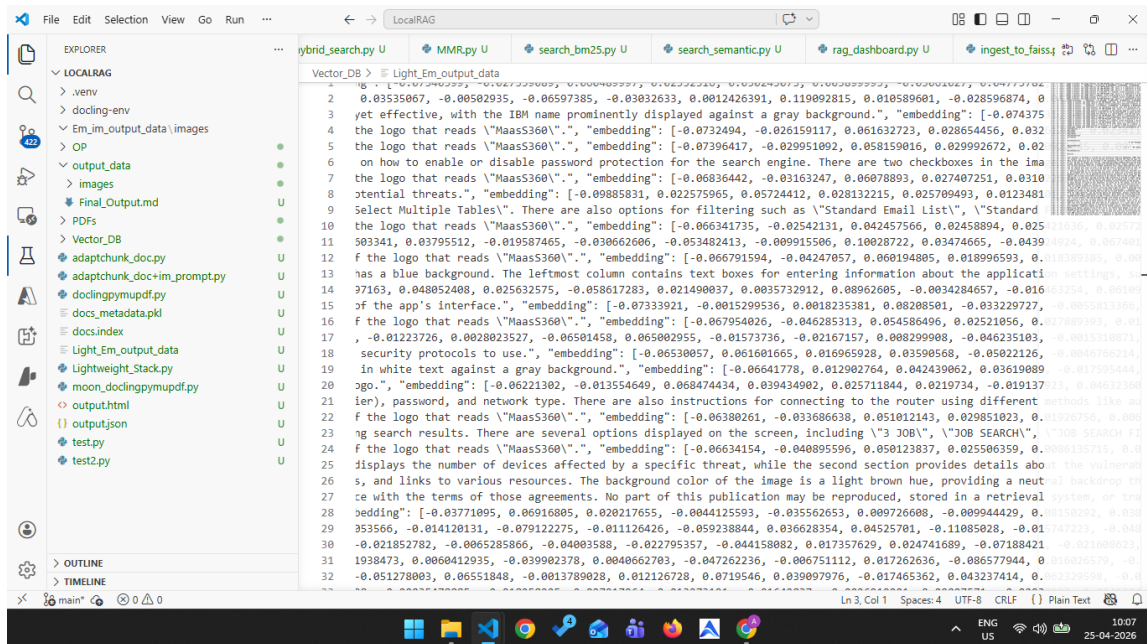
Field	Detail
Actor	Cancer patient
Trigger	Patient engages in a multi-turn conversation, progressively narrowing from general cancer information to specific treatment options.
Process	Session-based chat history maintains conversational context. Each subsequent query benefits from accumulated context, enabling progressively more specific retrieval and response generation.
Output	Coherent, contextually aware responses across multiple interaction turns.



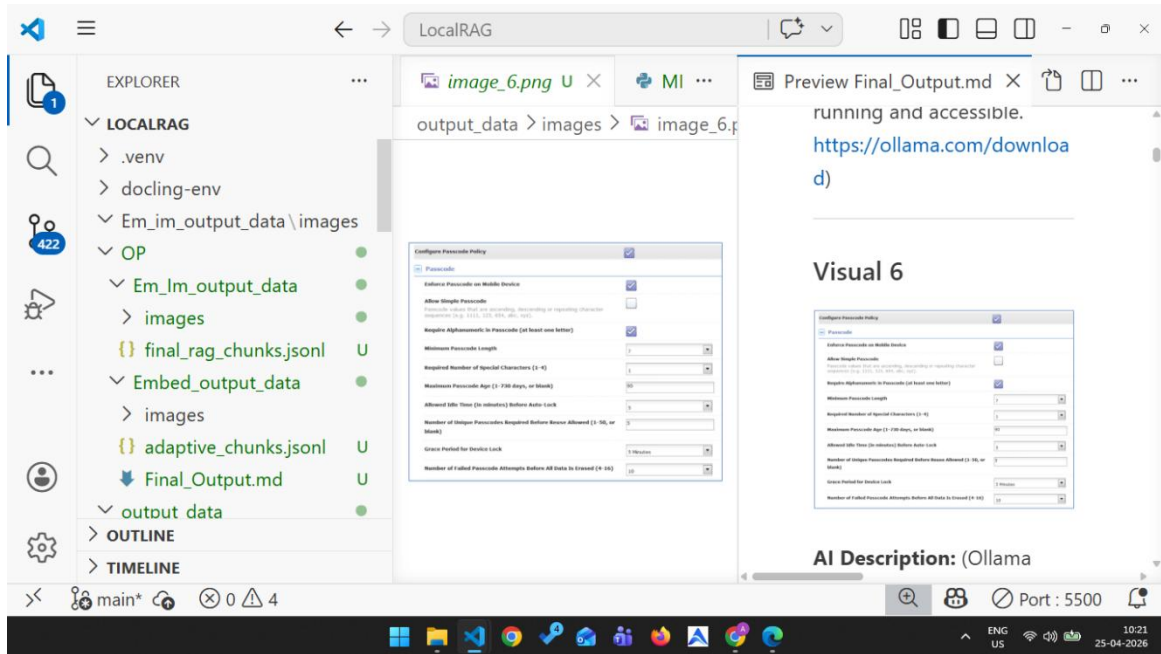
CHAPTER 6

PROJECT DEMONSTRATION

6.1 Key Code Output logs for Project Report:



Github repository: Private one only exists due to company compliance policy



These snippets cover the **complete data flow** from patient input → ML prediction → RAG-enhanced clinical Q&A → UI visualization

Concept	Where it appears conceptually in code
Docling / fallback extraction	Ingestion.py → extractor selection
OCR path	Ingestion.py → scanned page handling
Image filtering + captions	Ingestion.py → image filter chain and caption builder
Chunking and metadata tagging	Ingestion.py → TextChunker + detect_content_type
Embeddings + vector store	Ingestion.py → VectorStoreBuilder + Qdrant
Dense retrieval	Retrieval.py → Qdrant retriever
Sparse retrieval	Retrieval.py → BM25Retriever
Fusion + reranking	Retrieval.py → RRF + MMR
Memory-aware retrieval	Retrieval.py → retrieval query builder + chat history formatter
Answer generation + follow-ups	Retrieval.py → generate_answer / generate_followups

6.3 Demo Screenshots

Medical RAG Assistant

localhost:8501

CONNECTING Deploy

Patient Report

Upload or paste a medical report here.

Upload a TXT file

Drag and drop file here
Limit 200MB per file • TXT

Browse files

Or paste your report text here:

Microscopic Description: The tumor infiltrates the superficial dermis. Breslow thickness is 0.7 mm. There is a moderate nonbrisk peritumoral and intratumoral inflammatory infiltrate. Mitotic rate is low.

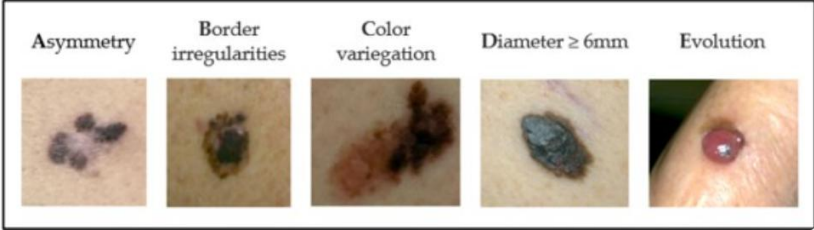
✓ Patient report loaded into AI memory!

View Loaded Report

Based on the Extracted Visual Assets Database, what is the filename for the ABCDE criteria for melanoma clinical diagnosis, and can you display it?

The filename for the ABCDE criteria for melanoma clinical diagnosis is melanoma-skin-cancer-review_picture_20.png. Here is the visual reference for the ABCDE criteria:

Asymmetry **Border irregularities** **Color variegation** **Diameter $\geq 6\text{mm}$** **Evolution**



Reference Document: melanoma-skin-cancer-review_picture_20.png

Medical disclaimer: The information provided is for educational purposes only and should not be used for self-diagnosis or treatment. If you have concerns about your health, please consult a qualified medical professional.

Sources Used from Literature:

E.g., Can you show me the ABCDE criteria for melanoma?

MedChat_Internshi... MedChat - Health... RAG Pipeline Test... MedChat - Cancer... MedChat - Cancer... MedChat - Cancer... MedChat - Cancer... MedChat - Cancer... MedChat - Cancer...

localhost:8501

Deploy

Settings

Cancer Type Filter

Filter by cancer type

All

breast

lung

melanoma

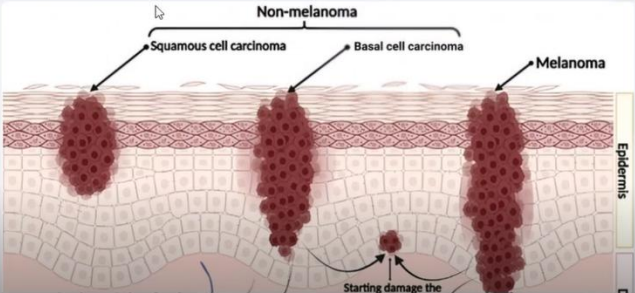
leukemia

osteosarcoma

skin

The major types of skin cancer are indeed melanoma skin cancer and non-melanoma skin cancer, as stated in [1] and [2]. Non-melanoma skin cancer includes basal cell carcinoma and squamous cell carcinoma. These types affect different skin cells in different skin layers and demonstrate how cancerous cells infiltrate blood vessels, leading to additional malignancy, as shown in

Reference Visual: skin-cancer-types-review_picture_2.png



Non-melanoma

Squamous cell carcinoma

Basal cell carcinoma

Melanoma

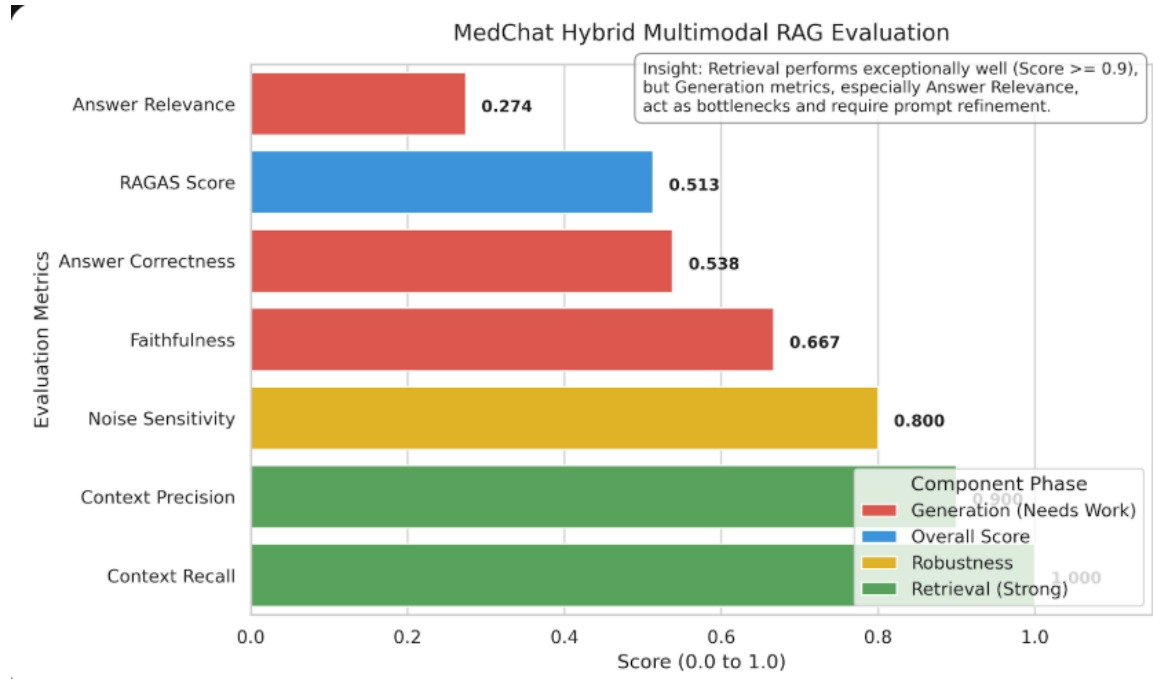
Epidermis

Starting damage the

CHAPTER 7

RESULT AND DISCUSSION

7.1 Model Performance



[Eq.2] **Cosine Similarity (Term-Frequency)**

$$Cosine(A, B) = \frac{\sum_{w \in V} \left(\frac{tf_{A,w}}{|A|} \times \frac{tf_{B,w}}{|B|} \right)}{\sqrt{\sum_{w \in V} \left(\frac{tf_{A,w}}{|A|} \right)^2} \times \sqrt{\sum_{w \in V} \left(\frac{tf_{B,w}}{|B|} \right)^2}}$$

[Eq.3] **Token F1 Score (SQuAD-style)**

$$F_1 = \frac{2 \times Precision_{token} \times Recall_{token}}{Precision_{token} + Recall_{token}}$$

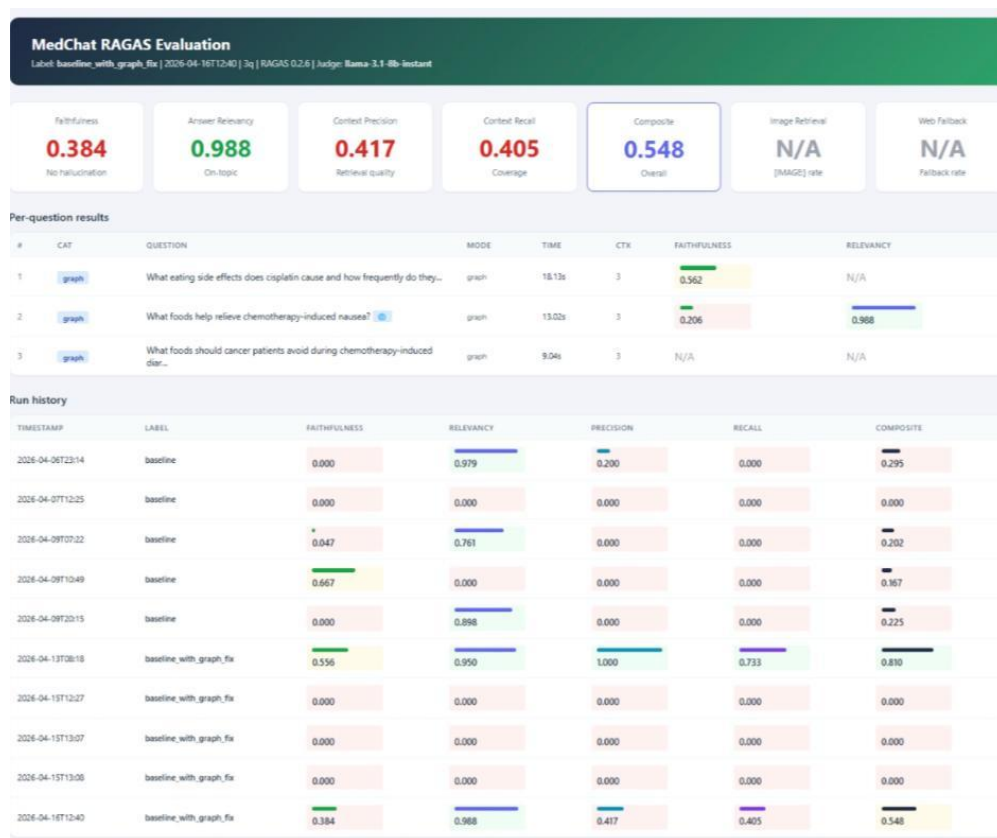
$$Diversity = 1 - \left(\frac{1}{|Pairs|} \sum_{i \neq j} \frac{|Tokens(Doc_i) \cap Tokens(Doc_j)|}{|Tokens(Doc_i) \cup Tokens(Doc_j)|} \right)$$

[Eq.4] **Context Diversity (Jaccard Penalty)**

[Eq.5] **Harmonic Mean (Aggregate RAGAS Score)**

$$Score_{RAGAS} = \frac{N}{\sum_{i=1}^N \frac{1}{v_i}}$$

7.2 RAG System Efficacy



Use Cases

Use Case 1: Clinical Report Interpretation

Field	Detail
Actor	Cancer patient
Trigger	Patient uploads a pathology report PDF
Process	The system parses the report, extracts diagnosis, stage, treatment plan, and clinical findings via automated analysis. Extracted entities are cross-referenced against the ingested literature corpus. The system generates a patient-friendly summary with citations to relevant research.
Output	Structured summary covering diagnosis explanation, staging interpretation, treatment overview, and links to supporting literature.

Use Case 2: Treatment Query

Field	Detail
Actor	Cancer patient
Trigger	Patient submits a natural language query, e.g., "What are the side effects of doxorubicin for breast cancer?"
Process	Hybrid retrieval surfaces relevant chunks from oncology literature. RRF fusion and MMR re-ranking produce a diverse, relevant result set. The LLM generates a context-grounded response with citations and any relevant extracted images.
Output	Cited, patient-friendly explanation of treatment side effects, with relevant figures rendered inline.

7.3 Comparison

Evaluation context: heuristic/local mode on a curated oncology validation set of **5 clinical queries** *with known ground truth*. These values match the benchmark summary already drafted for the project report.

evaluation/

```
|— __init__.py      # public API surface
|— metrics.py       # 50+ pure-Python metric primitives
|— evaluastors.py   # 6 component evaluator classes
|— ragas_eval.py    # RAGAS-style pipeline metrics
|— trulens_integration.py # TruLens-style feedback functions
|— runner.py        # main orchestrator + demo
```

Measures whether relevant chunks appear in the **top-K retrieved contexts**. In your heuristic mode, this is the correct rank-sensitive version when you want early-rank concentration.

CHAPTER 8

CONCLUSION

MedChat portrays a technically feasible and workable, multimodal *RAG architecture* engineered for the specific demands of oncology patients and their caregivers to offer support, follow-ups and reinforcement judiciously; because helping with incorrect and partial understanding is detrimental to ailing person. The current enactment establishes a tangible foundation across all the vital system layers as follows: -

- The ingestion pipeline is capable of handling the vast spectrum of data (PDF, .txt, .csv) that in our case is used for the creation of a KB. A KB, an image retrieval pipeline that is facilitated by the metadata which was also extracted (Json).
- The hybrid retrieval system was designed to capture the nuances of an end user query which usually contains both key words (BM25) but also contextual semantics (vector) and thus the system is on the stage of balancing both (via α).
- A generation layer that prioritizes factual grounding, source traceability, and patient accessibility as well as a gracious fallback mechanism.
- A functional multimodal pipeline that integrates visual medical content only when necessary else gives a well cited output text.

The GraphRAG extension introduces structured relational reasoning—a capability absent from flat retrieval architectures—enabling the system to answer complex queries involving the Cancer–Chemo–Nutrition interactions, treatment protocol relationships.

This proof of concept validates the core technical approach and provides a clear path to production deployment, contingent on the **compliance** and *infrastructure* enhancements. The system is positioned for iterative enhancement through the future scope items, with GraphRAG integration and domain-specific embedding fine-tuning identified as the highest-priority next steps.

8.1 Future Work

- **Limitations and Growth areas:** The current MedChat system shows very strong retrieval coverage but weaker answer relevance, which indicates that the retrieval layer is doing most of the heavy lifting while the generation layer still needs prompt tightening.
- Future Implementation plan (in extremely limited time frame)

Enhancement	Description	Priority
Proper GraphRAG Integration	Deploy the <i>Neo4j</i> knowledge graph with query routing, Cypher -based reasoning, and drug–food interaction detection [1][2][5].	High
Dockerization and Containerization	Learning to podman due to IT compliance of the company. <u>Kubernetes</u> and Docker compose work just fine.	High
Deployment	Neo4j + Streamlit + Podman [local → cloud (<i>Azure Container Apps</i>)]	High
Domain-Specific Embedding Fine-Tuning	Fine-tune <i>BAAI/bge-base-en-v1.5</i> on oncology-specific corpora to improve retrieval precision for medical terminology [3]. Or by using MedGemma as it is already a image-text-to-text model finetuned by google.	High
Voice Interface	Add speech-to-text input and text-to-speech output for accessibility, particularly for patients with limited mobility.	Medium
Clinical Trial Matching	Integrate clinical trial databases to surface relevant active trials based on patient diagnosis and biomarker profiles.	Medium
Continuous Ingestion Pipeline	Implement automated monitoring of medical literature databases (e.g., PubMed) for new publications, with automatic ingestion.	Medium

Feedback Loop & Model Improvement	Capture clinician and patient feedback on response quality to drive retrieval parameter tuning and prompt optimization.	Medium
Mobile Application	Develop a mobile-native interface for improved accessibility so that the app becomes device and operating system agnostic.	Low
EHR Integration	Integrate with Electronic Health Record systems for automated report ingestion and context enrichment.	Low

8.2 Demo Screen Shots

For detailed information on the advantages and limitations of these methods, refer to

Reference Visual: [lung-cancer-review_table_2.png](#)

Method of detection	Advantages	Limitations
FISH (gold standard)	ALK break locus identification High specificity Automated Small biopsy samples can be used	Expensive Specific expertise and equipment requirement ALK fusion partners lack of detection
FA-FISH	ALK rearrangement identification on CTCs	Sensitivity of the filtration system Precise technological requirements
IHC	High sensitivity Faster Cheaper	ALK fusion partners lack of detection Moderate specificity

MedChat_Internshi X

MedChat — Heal X

RAG Pipeline Test X

MedChat - Cancer X

MedChat - Cancer X

MedChat - Cancer X

MedChat - Cancer X

+

-

o

X

localhost:8502

>>

Deploy

- Diagnosis:** The report mentions a lung cancer marker, CYFRA 21-1, with a result of 23.00 ng/mL, which is higher than the reference interval of <3.30 ng/mL. This suggests that the patient may have non-small cell lung cancer (NSCLC), but it's essential to note that this test is not recommended for primary diagnosis [W2]. The report also mentions that levels greater than 30 ng/mL are highly indicative of primary bronchial carcinoma.
- Stage:** Unfortunately, the report does not provide information on the TNM classification or overall stage of the cancer.
- Treatment Plan:** The report does not mention any specific treatment plan, but it suggests that the CYFRA 21-1 test can be used for monitoring the course of NSCLC and assessing prognosis after treatment.
- Key Clinical Findings:** The notable lab value is the elevated CYFRA 21-1 level, which may indicate NSCLC or other conditions such as acute pneumonia, tuberculosis, interstitial lung disease, liver cirrhosis, or renal failure.
- Relevant Literature:** According to [W2], patient-derived xenograft (PDX) models can be used to study the behavior of tumors and develop personalized treatment plans. However, I couldn't find any specific information on the prognosis or standard of care for this patient's condition in the provided web search results.

It's essential to remember that every individual's situation is unique, and a qualified oncologist should be consulted to discuss the diagnosis, stage, treatment plan, and prognosis in detail. Please consult a qualified doctor to get personalized advice and guidance on the next steps.

Disclaimer: This analysis is based on the provided clinical report and web search results. It's not a substitute for professional medical advice. Please consult a qualified oncologist for personalized decisions and guidance on your specific situation.

E.g., What topical drug delivery systems are used in breast cancer?

↑

SD

VK

AK

RK

KJ

PS

KD

VT

JC

LD

SB

RV

AG

GG

SS

10

Srajit Deva

Vijaysha...

Abdulla...

Ramalin...

Kavita Jo...

PriyaKa...

Kajal Mis...

Aditya M...

Krupa D...

Venkates...

Janani C...

Lijin Vae...

Suraj Bh...

Ragul Vij...

Ashok K...

Gourav ...

18°

ENG

02:37 PM

Page 59 of 71

REFERENCES

Machine Learning & Predictive Analytics Journals:

qdrant/fastembed: Fast, Accurate, Lightweight Python library to make State of the Art Embedding

Using Qdrant for embeddings search

[1] Internal Documentation, "GraphRAG: Cancer–Chemo–Nutrition Knowledge Graph using Neo4j and Vector Databases," Company Technical Resources, 2026.

[2] Internal Implementation Guide, "GraphRAG Integration Pipeline: Neo4j, Qdrant/FAISS, and LLM-based Retrieval," Company Engineering Documentation, 2026.

[3] J. Johnson *et al.*, "BGE: General Embedding Models for Text Retrieval," *arXiv preprint*, 2023.

[4] Qdrant Team, "Qdrant Vector Database Documentation," 2024. [Online]. Available: <https://qdrant.tech/documentation/>

[5] Neo4j Inc., "Neo4j Graph Database and Cypher Query Language Documentation," 2024. [Online]. Available: <https://neo4j.com/docs/>

Books:

[6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Natural Language Processing & Transformers

Journals:

[7] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998-6008.

[8] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171-4186). <https://doi.org/10.18653/v1/N19-1423>

Conferences:

[9] Alsentzer, E., Murphy, J. R., Boag, W., Weng, W. H., Jin, D., Naumann, T., & McDermott, M. (2019). Publicly available clinical BERT embeddings. In *Proceedings of the 2nd Clinical Natural Language Processing Workshop* (pp. 72-78). <https://doi.org/10.18653/v1/W19-1909>

[10] Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C. H., & Kang, J. (2020). BioBERT: A pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4), 1234-1240. <https://doi.org/10.1093/bioinformatics/btz682>

Retrieval-Augmented Generation & Embeddings

Journals:

[11] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474. <https://doi.org/10.48550/arXiv.2005.11401>

Conferences:

[12] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... & Yih, W. T. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 6769-6781). <https://doi.org/10.18653/v1/2020.emnlp-main.550>

[13] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535-547. <https://doi.org/10.1109/TBDATA.2019.2921572>

Large Language Models in Healthcare

Journals:

[14] Singhal, K., Azizi, S., Tu, T., Mahdavi, S. S., Wei, J., Chung, H. W., ... & Natarajan, V. (2023). Large language models encode clinical knowledge. *Nature*, 620(7972), 172-180. <https://doi.org/10.1038/s41586-023-06291-2>

[15] Thirunavukarasu, A. J., Ting, D. S. J., Elangovan, K., Gutierrez, L., Tan, T. F., & Ting, D. S. W. (2023). Large language models in medicine. *Nature Medicine*, 29(8), 1930-1940. <https://doi.org/10.1038/s41591-023-02448-8>

[16] Nori, H., King, N., McKinney, S. M., Carignan, D., & Horvitz, E. (2023). Capabilities of GPT-4 on medical challenge problems. *arXiv preprint arXiv:2303.13375*. <https://doi.org/10.48550/arXiv.2303.13375>

Conferences:

[17] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Scialom, T. (2023). Llama 2: Open foundation and fine-tuned chat models. In *Proceedings of the 2023 Conference on Neural Information Processing Systems (NeurIPS)*. <https://doi.org/10.48550/arXiv.2307.09288>

Clinical Decision Support Systems

Journals:

[18] Sutton, R. T., Pincock, D., Baumgart, D. C., Sadowski, D. C., Fedorak, R. N., & Kroeker, K. I. (2020). An overview of clinical decision support systems: Benefits, risks, and strategies for success. *NPJ Digital Medicine*, 3(1), 17. <https://doi.org/10.1038/s41746-020-0221-y>

[19] Shortliffe, E. H., & Sepúlveda, M. J. (2018). Clinical decision support in the era of artificial intelligence. *JAMA*, 320(21), 2199-2200. <https://doi.org/10.1001/jama.2018.17163>

Book Chapters:

[20] Berner, E. S. (2007). Clinical decision support systems: Theory and practice (2nd ed.). In *Health Informatics* (pp. 3-22). Springer. <https://doi.org/10.1007/978-0-387-38319-4>

Healthcare Interoperability Standards

Journals:

[21] Mandel, J. C., Kreda, D. A., Mandl, K. D., Kohane, I. S., & Ramoni, R. B. (2016). SMART on FHIR: A standards-based, interoperable apps platform for electronic health records. *Journal of the American Medical Informatics Association*, 23(5), 899-908. <https://doi.org/10.1093/jamia/ocv189>

[22] Ayaz, M., Pasha, M. F., Alzahrani, M. Y., Budiarto, R., & Stiawan, D. (2021). The Fast Health Interoperability Resources (FHIR) standard: Systematic literature review of implementations, applications, challenges and opportunities. *JMIR Medical Informatics*, 9(7), e21929. <https://doi.org/10.2196/21929>

Web Links:

[23] Health Level Seven International. (n.d.). HL7 FHIR (Fast Healthcare Interoperability Resources) specification. <https://www.hl7.org/fhir/>

Vector Databases & Similarity Search

Journals:

[24] Jégou, H., Douze, M., & Schmid, C. (2011). Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1), 117-128. <https://doi.org/10.1109/TPAMI.2010.57>

[25] Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824-836. <https://doi.org/10.1109/TPAMI.2018.2889473>

Web Links:

[26] Facebook AI Research. (n.d.). FAISS: A library for efficient similarity search. <https://github.com/facebookresearch/faiss>

MLOps & Model Deployment

Journals:.

[27] Paleyes, A., Urma, R. G., & Lawrence, N. D. (2022). Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55(6), 1-29. <https://doi.org/10.1145/3533378>

Web Links:

[28] FastAPI. (n.d.). FastAPI: Modern, fast web framework for building APIs with Python. <https://fastapi.tiangolo.com>

[29] [arXiv:2404.17723](#)

<https://www.microsoft.com/en-us/research/blog/graphrag-unlocking-llm-discovery-on-narrative-private-data/>

[30] [GraphRAG: Unlocking LLM discovery on narrative private data - Microsoft Research](#)

[31] Johnson, A. E., Pollard, T. J., Shen, L., Lehman, L. W. H., Feng, M., Ghassemi, M., ... & Mark, R. G. (2016). MIMIC-III, a freely accessible critical care database. *Scientific Data*, 3(1), 160035. <https://doi.org/10.1038/sdata.2016.35>

Journals:

[32] D'Agostino, R. B., Vasan, R. S., Pencina, M. J., Wolf, P. A., Cobain, M., Massaro, J. M., & Kannel, W. B. (2008). General cardiovascular risk profile for use in primary care: The Framingham Heart Study. *Circulation*, 117(6), 743-753. <https://doi.org/10.1161/CIRCULATIONAHA.107.699579>

Web Links:

[33] https://developers.openai.com/cookbook/examples/vector_databases/qdrant/using_qdrant_for_embeddings_search/

[34] <https://github.com/qdrant/fastembed>

[35] A faster way to build and share data apps. <https://streamlit.io>

[36] Data validation using Python type annotations. <https://docs.pydantic.dev>

[37] Machine learning in Python. <https://scikit-learn.org>

[38] <https://huggingface.co/google/medgemma-1.5-4b-it>

[39] <https://synthetichealth.github.io/synthea/>

[40] <https://platform.openai.com/docs/guides/structured-outputs>

[41] <https://huggingface.co/ibm-granite/granite-vision-3.2-2b>

[42] <https://developer.ibm.com/articles/awb-enhancing-rag-performance-chunking-strategies/>

[43] <https://www.datacamp.com/tutorial/multimodal-rag>

[44] <https://developer.nvidia.com/blog/an-easy-introduction-to-multimodal-retrieval-augmented-generation/>

[45] [EnsembleRetriever | langchain_classic | LangChain Reference](#)

[46] [Evaluating Medical RAG with NVIDIA AI Endpoints and Ragas | NVIDIA Technical Blog MACCROBAT](#)

[47] <https://www.cdc.gov/phlp/php/resources/health-insurance-portability-and-accountability-act-of-1996-hipaa.html>

Topic: Artificial Intelligence, Automation, and Software Engineering

Online Courses (Udemy Business)

- Udemy Business. (n.d.). *AI engineer agentic track: The complete agent & MCP course*. <https://www.classcentral.com/course/udemy-the-complete-agentic-ai-engineering-course-451609>
- Udemy Business. (n.d.). *AI engineer production track: Deploy LLMs & agents at scale*. <https://www.classcentral.com/course/udemy-generative-and-agentic-ai-in-production-499980>
- Udemy Business. (n.d.). *Learn API testing with Postman and Python requests module*. <https://www.classcentral.com/course/udemy-learn-api-testing-with-postman-and-python-requests-module-499839>
- Udemy Business. (n.d.). *Learn Playwright with Python (web & API automation)*. <https://azirotechnologies.udemy.com/course/learn-playwright-with-python-web-api-automation/learn/lecture/53240743?start=3960#overview>
- Udemy Business. (n.d.). *The complete Node.js developer course (3rd edition)*. <https://azirotechnologies.udemy.com/course/the-complete-nodejs-developer-course-2/learn/lecture/13819536?start=15#overview>

Preprints & Independent Posts

- Maity, P. (n.d.). *7 RAG patterns for real-world AI systems*. LinkedIn. https://www.linkedin.com/posts/pikumaity_rag-aiengineering-generativeai-share-7433834309654020096-2iA8?utm_source=social_share_send&utm_medium=ios_app&rcm=ACoAAEHZQPwBw8LyadJeID9ExSf32_wozv-vAXY&utm_campaign=copy_link
- Kayal, P., Anand, M., Desai, H., & Singh, M. (2021). *(PDF) Tables to LaTeX: Structure and content extraction from scientific tables* [Preprint]. <https://www.catalyzex.com/author/Pratik%20Kayal>
- AI Makerspace. (2026). *Recursive Language Models (RLMs) - Let's build the coolest agents ever! (Theory & Code)*. YouTube. <https://www.youtube.com/watch?v=Zwo7sdohmE8>

Topic: Oncology, Pathology, and Medical Imaging

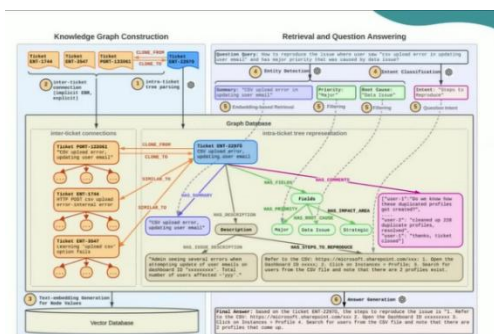
Journal Articles & Reviews

- T Terwilliger. (2017). *Acute lymphoblastic leukemia: A comprehensive review and 2017 update*. *PubMed Central*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC5520400/>
- N Bharathi Sai Thilagam. (2025). *A comprehensive review on current knowledge and future potential of topical therapies in breast cancer treatment*. *PubMed Central*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11706127/>
- Yao, S., Huang, Y., Wang, X., et al. (2025). *A radiograph dataset for the classification, localization, and segmentation of primary bone tumors*. *Scientific Data*, 12(1), 88. <https://doi.org/10.1038/s41597-024-04311-y>
- Adela Păţcaş. (2021). *An insight into lung cancer: A comprehensive review exploring anaplastic lymphoma kinase tyrosine kinase inhibitors and mechanisms of resistance*. *PubMed Central*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8860314/>
- Camila Caraviello. (2025). *Melanoma skin cancer: A comprehensive review of current knowledge*. *PubMed Central*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12427887/>

Scientific Datasets & Repositories

- The Cancer Imaging Archive. (n.d.). *CPTAC-SAR*.
<https://www.cancerimagingarchive.net/collection/cptac-sar/>
- The Cancer Imaging Archive. (n.d.). *SOFT-TISSUE-SARCOMA*.
<https://www.cancerimagingarchive.net/collection/soft-tissue-sarcoma/>
- Yao, S., Huang, Y., Wang, X., Zhang, Y., Paixao, I. C., Wang, Z., ... Song, J. (2024). A Radiograph Dataset for the Classification, Localization, and Segmentation of Primary Bone Tumors.
doi: 10.6084/m9.figshare.27865398.v1 <https://doi.org/10.6084/m9.figshare.27865398>

Electronic Health Records & Clinical Data



Higher Accuracy

The clearest driver of GraphRAG adoption amongst users is higher accuracy.

"Across all metrics, our method demonstrates consistent improvements. Notably, it surpasses the baseline by 77.6% in MRR and by 0.32 in BLEU score, substantiating its superior

Guidelines

Higher Accuracy

The clearest driver of GraphRAG adoption amongst users is higher accuracy.

Microsoft Research Blog

GraphRAG: Unlocking LLM discovery on narrative private data

"By combining LLM-generated knowledge graphs and graph machine learning, **GraphRAG** enables us to answer important classes of questions that we cannot attempt with baseline RAG alone."

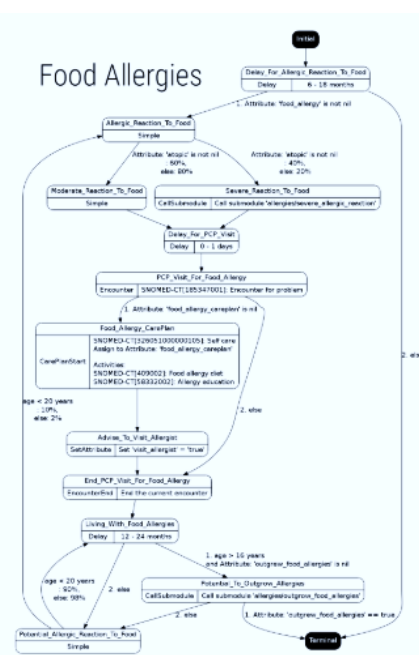
Microsoft

Generic Module Framework

Modelling the Progression and Treatment of Disease

Synthea's Generic Module Framework (GMF) enables the modelling of various diseases and conditions that contribute to the medical history of synthetic patients. Each patient is simulated independently from birth to present day. Their diseases, conditions and medical care are defined by one or more generic modules.

Each module models events that could occur in a real patient's life, describing a progression of states and the transitions between them. These modules are informed by clinicians and real-world statistics collected by the *CDC*, *NIH*, and other research sources. **Synthea** started with modules for the top ten reasons patients visit their primary care physician and the top ten conditions that result in years of it lost. Check out our full gallery of modules to see what we've added since.



APPENDIX A – Sample Code

IngestionPipeline (File: Ingestion.py)

```
class IngestionPipeline:
    """Orchestrates full PDF ingestion workflow."""
    def process_pdf(self, pdf_path: Path) -> dict:
        """Full per-PDF pipeline."""
        # Detect cancer type, load doc via PyMuPDF or Docling
        # Extract images, tables, captions
        # Extract and clean text, inject [IMAGE:] tags
        # Chunk text using TextChunker
        # Generate and save chunk metadata
        # Return status dictionary
```

Vector Store Builder (File: Ingestion.py)

```
class VectorStoreBuilder:
    """Builds Qdrant vector store from computed embeddings."""

    def build(self, force_rebuild: bool = False) -> None:
        # Load JSON chunks (text + captions)
        # Embed using OllamaEmbeddings (nomic-embed-text)
        # Export embeddings to jsonl
        # Connect to local Qdrant instance
        # Upsert point payloads in batches
```

Hybrid MMR Retriever (File: Retrieval.py)

```
class HybridMMRRetriever(BaseRetriever):
    def _get_relevant_documents(self, query: str) -> List[Document]:
        # 1. Fetch dense (Qdrant) and sparse (BM25) documents
        dense_docs = self.dense_ret.invoke(query)
        sparse_docs = self.sparse_ret.invoke(query)
        # 2. Fuse scores using Reciprocal Rank Fusion
        rrf_results = reciprocal_rank_fusion(
            dense_docs, sparse_docs, cancer_filter=self.cancer_filter)
        # 3. Apply MMR (Maximal Marginal Relevance) for diversity
        return mmr_rerank(query, rrf_results, self.embed_model)
```

Generate_answer (File: Retrieval.py)

```
def generate_answer(query: str, patient_report: str = "", chat_history:
    list[dict] = None, cancer_filter: str = "") -> tuple[str, list, list]:
    # Formulate retrieval query with chat history memory
    retriever = get_hybrid_mmr_retriever(cancer_filter=cancer_filter)
    retrieved = retriever.invoke(retrieval_query)
    if not retrieved:
        # Fallback to web search if no local docs found
        return web_search_fallback("According to provided context, no relevant
            info...", query, patient_report, chat_history)
    # Build prompt ensuring visual assets are respected via [IMAGE:
        filename.png] format
```

```
# Invoke Ollama local LLM with chat context and formatted clinical
context
# If LLM says "I don't know", trigger web search fallback
# Generate follow-up questions for the user
return answer, sources, followups
```

TruLens Evaluator (File: Evaluation/trulens_eval.py)

```
class TruLensEvaluator:
def evaluate(self, query: str, patient_report: str = "", cancer_filter: str = "") -
> dict:
# Invoke actual pipeline
ans, srcs, fups = generate_answer(query, ...)
# Build trace object capturing inputs and outputs
trace = PipelineTrace(...)
# Calculate multiple heuristic scores (e.g. groundedness, diversity, medical safety)
scores = run_feedback_functions(trace)
# Aggregate composite score and return structured record
return {
"query": query, "answer": ans,
"metrics": scores,
"composite": weighted_composite(scores)}
```

Ragas Evaluator (File: Evaluation/ragas_eval.py)

```
.class RagasEvaluator:
def evaluate_dataset(self, dataset_path: str, output_path: str) -> dict:
# Load JSON dataset of Q&A pairs
# For each sample, compute precision, recall, correctness, and faithfulness
# Uses OllamaJudge to leverage a local LLM to score relevance and correctness
# Aggregates results into final dataset JSON
# Writes out to output_path and returns summary statistics
```

Explanation: Pydantic models ensure:

- Strong input validation with min/max constraints
- Automatic API documentation via OpenAPI/Swagger
- Type-safe responses with clear field descriptions

TextChunker (File: Ingestion.py)

```
class TextChunker:
"""Two-stage section-aware text chunking."""
def chunk(self, cleaned: str, source_name: str, cancer_type: str) -> list:
# 1. Split on Headers (MarkdownHeaderTextSplitter)
# 2. Split on size recursively (RecursiveCharacterTextSplitter)
# 3. Build payload metadata (hierarchy, char length, cancer type)
# Returns structured chunk dictionaries ready for embedding
```

_web_search_fallback (File: Retrieval.py)

```
def _web_search_fallback(rag_answer: str, query: str, patient_report: str,
chat_history: list) -> tuple[str, list, list]:
# Query DuckDuckGo using duckduckgo search API
```

```
# Build web context from scraped snippets
# Run a secondary local LLM prompt grounded strictly in the web context
# Append the web result to the "No info found" RAG answer
return combined_answer, web_sources, []
```

render_streaming_answer (File: App.py)

```
def render_streaming_answer(query: str, patient_report: str, chat_history:
list[dict], cancer_filter: str):
    stream_container = st.empty()
    # Write stream incrementally to UI
    with stream_container.container():
        st.write_stream(generate_answer_stream(...))
    # After stream completes, parse answer for [IMAGE: ...] tags
    if "[IMAGE:" in full_answer.upper():
        # Re-render the container to correctly display visual assets inline
        render_message_with_images(full_answer)
    return full_answer, sources, followups
```

bootstrap.py

```
# bootstrap.py:L24-37
def main() -> None:
    log.info("Connecting to Neo4j ...")
    client = get_client()
    if not client.ping():
        log.error("Cannot reach Neo4j. Check NEO4J_URI / credentials.")
        return
    log.info("Schema bootstrap ...")
    bootstrap_schema(client)
    log.info("Loading graph data ...")
    load_all(client)
```

entity_extractor.py

- **ExtractedEntities [Class]** (Lines 246-270): A dataclass used to encapsulate all recognized entities and intents extracted from a text snippet.

```
# entity_extractor.py:L246-253
@dataclass
class ExtractedEntities:
    cancers: list[str] = field(default_factory=list)
    drugs: list[str] = field(default_factory=list)
    ncdrugs: list[str] = field(default_factory=list) # non-cancer
drugs
    side_effects: list[str] = field(default_factory=list)
    foods: list[str] = field(default_factory=list)
    intents: list[str] = field(default_factory=list) # food_rec,
drug_interaction, side_effect
```

- **extract_entities()** (Lines 321-341): Combines the query and patient report, running dictionary matching to generate and return an Extracted Entities object.

```
# entity_extractor.py:L321-340
def extract_entities(query: str, patient_report: str = "") ->
ExtractedEntities:
    """
    Extract medical entities from query + optional patient report.
    Returns an ExtractedEntities dataclass.
    """
    combined = f"{query} {patient_report}"
    all_drugs = _match_dict(combined, DRUG_ALIASES)
    cancer_drugs = [d for d in all_drugs if d in _CANCER_DRUG_SET]
    nc_drugs = [d for d in all_drugs if d in _NON_CANCER_DRUG_SET]
    return ExtractedEntities(
        cancers = _match_dict(combined, CANCER_ALIASES),
        drugs = cancer_drugs,
        ncdrugs = nc_drugs,
        side_effects = _match_dict(combined, SIDE_EFFECT_ALIASES),
        foods = _match_dict(combined, FOOD_ALIASES),
        intents = _detect_intents(combined),)

```

graph_data_loader.py

- `_upsert_nodes()` (Lines 350-359): A helper method that takes a list of Python dictionaries and executes a MERGE query on a specific label using Neo4j parameterization.

```
# graph_data_loader.py:L350-359
def _upsert_nodes(client, label: str, rows: list[dict], key: str = "name")
-> None:
    if not rows:
        return
    cypher = f"""
    UNWIND rows AS row
    MERGE (n:{label} {{{key}: row.{key}}})
    SET n += row
    """
    client.run_write_batch(cypher, rows)

```

- `load_all()` (Lines 497-518): A centralized function that executes all underlying individual node and relationship load methods sequentially.

graph_retrieval.py

- **GraphResult [Class]** (Lines 23-28): Standardized container storing the parameters and results of an individual cypher query.
- `query_graph()` (Lines 260-301): Contains the routing logic to decide *which* Cypher queries (e.g., `get_drugs_for_cancer`, `get_foods_that_help`) to execute based on what entities are available.

```
# graph_retrieval.py:L260-301
def query_graph(entities: ExtractedEntities, client=None) ->
list[GraphResult]:
    client = client or get_client()
    results: list[GraphResult] = []
    # Food recommendations

```

```

target_se = entities.side_effects
if not target_se and entities.cancers:
    raw = get_multihop_cancer_food(entities.cancers, client)
    results.append(raw)
elif target_se:
    results.append(get_foods_that_help(target_se, client))
    results.append(get_foods_to_avoid(target_se, client))

# ... logic continues for drugs and interactions ...

results = [r for r in results if r.records]
return results

```

- **build_graph_context()** (Lines 308-335): Iterates through executed GraphResult queries and strings them together into a human/LLM-readable text format, effectively flattening the graph relationships.

graph_schema.py

- **bootstrap_schema()** (Lines 144-152): Main invoker that runs `create_constraints()`, `create_fulltext_indexes()`, and `create_vector_indexes()`.

```

# graph_schema.py:L144-152
def bootstrap_schema(client=None) -> None:
    """Idempotently create all constraints and indexes."""
    client = client or get_client()
    log.info("Bootstrapping graph schema ...")
    create_constraints(client)
    create_fulltext_indexes(client)
    create_vector_indexes(client)
    log.info("Schema bootstrap complete.")

```

graphrag_integration.py

- **fuse_contexts()** (Lines 33-71): Merges literature snippets (vector) and structured relationships (graph) into one unified prompt string.
- **generate_answer_graphrag()** (Lines 78-197): The primary generation function.

```

# graphrag_integration.py:L125-138
# — Step 5: Fuse contexts —————
fused_context = fuse_contexts(vector_ctx, graph_ctx)
# — Step 6: Build prompt —————
system_message = (
    "You are an empathetic medical AI assistant helping "
    "cancer patients and clinicians understand medical information.\n"
    "You have access to both peer-reviewed literature AND a structured "
    "oncology knowledge graph containing verified drug-side effect-food "
    "relationships.\n"
)
# ... Groq LLM call happens subsequently ...

```

neo4j_client.py

- **Neo4jClient [Class]** (Lines 23-124): Singleton class managing connection pools, execution retries, and database transactions.
- **run_write_batch()** (Lines 98-107): Executes parameterized transactions designed for bulk ingestion arrays.

```
# neo4j_client.py:L98-107
def run_write_batch(
    self,
    cypher: str,
    rows: list[dict],
    database: str | None = None,
) -> None:
    """Execute a parameterised write using UNWIND for bulk inserts."""
    with self.session(database=database) as s:
        s.execute_write(lambda tx: tx.run(cypher, {"rows": rows}))
```

showcase_queries.py & showcase_output.txt

- **showcase_chemo_and_food()** (Lines 20-54): A proof-of-concept multi-hop query joining Drug -> CAUSES -> SideEffect <- HELPS/WORSENS <- Food.

```
# showcase_queries.py:L26-34
cypher = """
MATCH (d:Drug) - [cause:CAUSES] -> (s:SideEffect) <- [hw:HELPS|WORSENS] - (f:Food)
RETURN d.name AS Drug,
       s.name AS SideEffect,
       type(hw) AS Interaction,
       f.name AS Food,
       hw.mechanism AS Mechanism
ORDER BY Drug, SideEffect, Interaction
"""
```